Todo list

Chapter Layout . . . 1
derive bounds for arm7 . . . 1
pseudo code for RRT*-GBO . . . 1
pseudo code for MC . . . 1
pseudo code for PHO . . . 1
pseudo code for IPSO . . . 1
iterative violation detector/via point cluster insertion . . . 1
Analytical Proof of optimal result for unconstrained SO3 maneuver . . . 1
Verification of SO3 result with 1 EZ (trajectory tangeant to the EZ) . . . 1
Kinematics/dynamics of SO3 . . . 1
Kinematics/dynamics of Arm7 . . . 1
Quaternion notation . . . 1
quaternion description . . . 1
axis-angle representation description . . . 1
denavit-hartenberg parameters . . . 1
example of blue todo . . . 1
example of green todo . . . 1
3.1: Cover SO(3) parameterizations with quaternions, MRPs, and axis-angle representations . . . 2
3.1: Detail conversions, singularities, and representation choices for optimization versus sampling . . . 2
3.1: Define the Modified (Craig) Denavit-Hartenberg convention for the 7-DOF manipulator . . . 2
3.2: Formulate spacecraft attitude dynamics on SO(3) . . . 2
3.2: Formulate 7-DOF robotic manipulator dynamics . . . 2
3.2: Define state, input, and kinodynamic constraint envelopes for both systems 2
3.2: Specify actuator, joint, angular-velocity, and angular-acceleration limits . . 2
3.3: Formalize the continuous-time optimal control problem . . . 2
3.3: Transcribe the OCP into a discrete nonlinear program . . . 2

3.3: Detail B-spline parameterization of state and control trajectories 2
3.3: Describe the IPOPT interface, discretization, and constraint handling . . . 2
3.4: Present MC-GBO, RRT*-GBO, P-HO, and IPSO consistently 2
3.4: Describe naive versus uniform SO(3) sampling 2
3.4: Define Arm7 sampling bounds 2
3.5: Detect kinodynamic and boundary violations along B-spline trajectories . 2
3.5: Locally densify time-grid nodes to enforce feasibility 2
3.6: Prove and analytically verify the unconstrained SO(3) maneuver baseline . 2
Write a short intro to this section 3
maybe include the EOMs for the MRP formulation??? 16
Edit this to not have set number of CVs and time 26
Write MC-GBO details 32
Write RRT*-GBO details 32
informed pseudocode in progress 33
Write PHO details 34
Write IPSO details 35

Chair of Sensor Based Robotic Systems and Intelligent Assistance Systems

Technical University of Munich

Ryan Lacey

Comparative Performance Analysis of RRT*-GBO and Global Solvers for Constrained Nonlinear Trajectory Optimization in Robotics and Aerospace

Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Science in Robotics, Mechatronics, and Biomechanical Engineering at the TUM School of Engineering and Design

Supervisor:	Prof. Dr. Alin Albu-Schäffer Chair of Sensor Based Robotic Systems and Intelligent Assistance Systems
Advisor:	Dr. Roberto Lampariello; Arne Sachtler, M.Sc.
Submitted by:	Ryan Lacey
Submission date:	2 November 2026 in Garching bei München

Task Description

Compare the Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO) algorithm with global solvers for constrained nonlinear trajectory optimization:

- Perform a literature review of kinodynamic motion planning algorithms with applications in robotics and aerospace.
- Utilize, verify, and adapt the DLR designed RRT*-GBO algorithm.
- Implement several other State-of-the-Art (SOTA) global solvers from the literature.
- Apply the implemented algorithms to three systems to evaluate their performance: 1. an special orthogonal group in three dimensions (SO(3)) spacecraft slew maneuver, 2. a 7 degree-of-freedom (degree of freedom (DOF)) robot manipulator, 3. an under-actuated aircraft (e.g. ducted fan).
- Implement dynamic (force and torque), kinematic (position, velocity, and acceleration), and spatial constraints (obstacle avoidance) for the evaluated systems.
- Evaluate and compare the performance of the RRT*-GBO algorithm and the implemented global solvers based on criteria such as computational efficiency, solution quality, and robustness.
- Document the findings and provide recommendations for the use of RRT*-GBO and global solvers in constrained nonlinear trajectory optimization for robotics and aerospace applications for future comparisons against machine learning-based approaches.

Abstract

Please provide an abstract in English here: `./auxfiles/abstractEN.tex`

Acknowledgments

I would like to thank the following for their support during my thesis: my advisor and supervisor, Dr. Roberto Lampariello, for the opportunity to pursue my thesis under his oversight at the German Aerospace Center (DLR) and his guidance, insight, and debates throughout the course of my research; to Dr. Caroline Specht for her assistance with the legacy RRT*-GBO code; to my colleagues and fellow thesis students at the DLR for their comradery throughout the past year both in and away from the office; and finally to my dog Timber for keeping me company on days at home and forcing me to take breaks to enjoy the sun outside.

Contents

Task Description **v**

Abstract **vii**

1 Methodology **3**

1.1 Mathematical Foundations & Representations 3

1.1.1 Attitude Representations 3

1.1.2 Manipulator Kinematics 11

1.2 B-Splines 12

1.3 Physical Models, Kinematics, Dynamics, & Constraints 14

1.3.1 Spacecraft Model 16

1.3.2 7-DOF Robotic Manipulator Model 22

1.4 Optimal Control Problem (OCP) & Nonlinear Programming (NLP) Formulation 28

1.5 Optimal Control Problem 29

1.5.1 Problem 1: Spacecraft Attitude Slew Maneuver 29

1.5.2 Problem 2: 7-DOF Manipulator Ball-Catching Maneuver 29

1.5.3 NLP Decision Variable Structure 30

1.5.4 NLP Optimization Framework 31

1.6 Algorithm Frameworks 31

1.7 Algorithm Frameworks 31

1.7.1 Monte Carlo Gradient-Based Optimization (MC-GBO) 32

1.7.2 Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO) 32

1.7.3 Probabilistic Homotopy Optimization (P-HO) 34

1.7.4 Inverse Dynamics Particle Swarm Optimization (IPSO) 35

1.8 Adaptive Constraint Refinement 36

1.9 Analytical Verification 36

A Notation **37**

A.1 Special operators and symbols 37

A.2 Abbreviations and acronyms 38

List of Algorithms **41**

List of Figures **43**

List of Tables **45**

References **48**

Chapter Layout

derive bounds for arm7

pseudo code for RRT*-GBO

pseudo code for MC

pseudo code for PHO

pseudo code for IPSO

iterative violation detector/via point cluster insertion

Analytical Proof of optimal result for unconstrained SO3 maneuver

Verification of SO3 result with 1 EZ (trajectory tangeant to the EZ)

Kinematics/dynamics of SO3

Kinematics/dynamics of Arm7

Quaternion notation

quaternion description

axis-angle representation description

denavit-hartenberg parameters

example of blue todo

example of green todo

3.1: Cover SO(3) parameterizations with quaternions, MRPs, and axis-angle representations

3.1: Detail conversions, singularities, and representation choices for optimization versus sampling

3.1: Define the Modified (Craig) Denavit-Hartenberg convention for the 7-DOF manipulator

3.2: Formulate spacecraft attitude dynamics on SO(3)

3.2: Formulate 7-DOF robotic manipulator dynamics

3.2: Define state, input, and kinodynamic constraint envelopes for both systems

3.2: Specify actuator, joint, angular-velocity, and angular-acceleration limits

3.3: Formalize the continuous-time optimal control problem

3.3: Transcribe the OCP into a discrete nonlinear program

3.3: Detail B-spline parameterization of state and control trajectories

3.3: Describe the IPOPT interface, discretization, and constraint handling

3.4: Present MC-GBO, RRT*-GBO, P-HO, and IPSO consistently

3.4: Describe naive versus uniform SO(3) sampling

3.4: Define Arm7 sampling bounds

3.5: Detect kinodynamic and boundary violations along B-spline trajectories

3.5: Locally densify time-grid nodes to enforce feasibility

3.6: Prove and analytically verify the unconstrained SO(3) maneuver baseline

Chapter 1

Methodology

Write a short intro to this section

1.1 Mathematical Foundations & Representations

1.1.1 Attitude Representations

The attitude of a rigid body in three dimensional space (3-D) space can be represented using a myriad of parameterizations, each with its own advantages and limitations. Common representations include rotation matrices, quaternions, Modified Rodrigues Parameters (MRPs), Euler angles, and axis—angle representations. Rotation matrices are a classic choice for representing attitudes offering a singularity-free, intuitive geometric interpretation, with a linear mapping of vectors. However, they are over-parameterized with nine elements versus the minimal three required to describe rotations. This increases the computational load. Quaternions offer a more compact, four-element singularity-free representation of rotations compared to rotation matrices, but suffer from double cover and an unintuitive physical meaning of the parameters. MRPs are a three-element representation—making them a minimal and efficient description of rotations—that avoid the double-cover issue of quaternions but contain singularities at 360 degrees. Euler angles are also a minimal representation with very intuitive geometric interpretation, but they suffer from gimbal lock. Finally, axis—angle representations describe rotations using a rotation axis and an angle about that axis. They provide a clear geometric interpretation and are minimal, but they become singular at zero rotation whereby the axis-of-rotation becomes ill-defined.

Coordinate Systems

To describe the attitude of a rigid body, it is first essential to describe the coordinate systems involved. Throughout this description, we consider two coordinate systems:

- **Inertial frame:** A reference frame fixed in space, notated by the subscript **i**. The inertial frame serves as the global reference frame upon which all other reference frames, positions, and orientations are defined in relation to.

- **Body frame:** A coordinate system fixed to the rigid body, denoted by the subscript **b**, used to describe the orientation of the body relative to itself.

Vectors are expressed with respect to a specific coordinate system, with the preceding superscript indicating the frame the vector is expressed in, e.g., ${}^{\mathbf{i}}\mathbf{x}$ denotes a vector $\mathbf{x}$ expressed in the inertial frame.

While in principle, the inertial frame is fixed in space, in practice a convenient approximation is to consider merely a very slowly moving frame, such as one attached to the Earth or center of the Solar system.

Rotation Matrices

A rotation matrix—also known as a direction cosine matrix (DCM) [11]—is a matrix which, when multiplied with a vector expressed in one coordinate frame, transforms it into another coordinate frame. Alternatively, it can be seen as an operator that rotates a vector while preserving its length [5]. The former description describes a *passive* rotation, where the coordinate frame is rotated relative to a fixed vector, while the latter describes an *active* rotation, where the vector itself is rotated within a fixed coordinate frame. The passive rotation conventions will be adopted throughout this work. A second important convention is also important to clarify: This work defines these matrices as mapping the body frame to the inertial frame—as opposed to the other way around. An example of the relative orientation of the inertial and body frames is shown in Fig. 1.1.

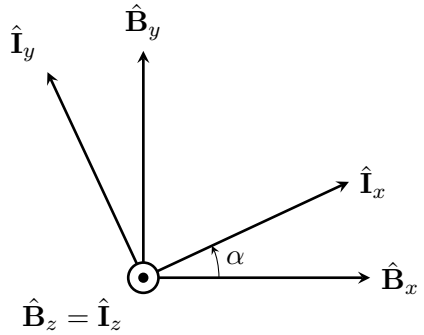


Figure 1.1: Rotation of the body frame $\{\hat{\mathbf{B}}_x, \hat{\mathbf{B}}_y, \hat{\mathbf{B}}_z\}$ by angle α about the z-axis to the inertial frame $\{\hat{\mathbf{I}}_x, \hat{\mathbf{I}}_y, \hat{\mathbf{I}}_z\}$.

Rotation matrices are the core mathematical representation used in this work to perform coordinate transformations of vectors, points, and coordinate frames. As such, a brief overview of their properties and usage is provided below.

The set of all proper orthogonal rotation matrices forms the *special orthogonal group* SO(3):

$$\mathrm{SO}(3) = \left\{ \mathbf{R} \in \mathbb{R}^{3\times 3} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1 \right\}. \tag{1.1}$$

$$\mathbf{R}^\top \mathbf{R} = \mathbf{I} \quad \Longleftrightarrow \quad \mathbf{R}^\top = \mathbf{R}^{-1}. \tag{1.2}$$

The elements of rotation matrices are referenced by their row and column indices:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}. \tag{1.3}$$

Vector coordinate transformations are performed by premultiplying the *vector* $\mathbf{v} \in \mathbb{R}^3$ by a rotation matrix:

$$^{\mathrm{i}}\mathbf{v} = \mathbf{R}_{\mathrm{ib}}\,{}^{\mathrm{b}}\mathbf{v} \tag{1.4}$$

Here the vector ${}^{\mathrm{b}}\mathbf{v}$ originally expressed in the body frame is transformed into the inertial frame ${}^{\mathrm{i}}\mathbf{v}$ using the rotation matrix $\mathbf{R}_{\mathrm{ib}}$. Equation (1.4) highlights that rotation matrices provide a direct linear mapping of vectors via simple matrix multiplication.

Unlike vector transformations, transforming a *point* $\mathbf{x} \in \mathbb{R}^3$ requires both rotation and translation, as points have a fixed position in space rather than just a direction and magnitude. To demonstrate, consider a point ${}^{\mathrm{b}}\mathbf{x}$ expressed in the body frame, the origin of the body frame being ${}^{\mathrm{b}}\mathbf{x}_{\mathrm{b}}$, and the origin of the inertial frame being ${}^{\mathrm{i}}\mathbf{x}_{\mathrm{i}}$. Its coordinates in the inertial frame ${}^{\mathrm{i}}\mathbf{x}$ are obtained by applying both the rotation and translation:

$$^{\mathrm{i}}\mathbf{x} = \mathbf{R}_{\mathrm{ib}}\left({}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{b}}\mathbf{x}_{\mathrm{b}}\right) + {}^{\mathrm{i}}\mathbf{x}_{\mathrm{i}} = \mathbf{R}_{\mathrm{ib}}\,{}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{i}}\mathbf{x}_{\mathrm{b}} + {}^{\mathrm{i}}\mathbf{x}_{\mathrm{i}} \tag{1.5}$$

Naturally, the origins of the body and inertial frames expressed in their respective frames are zero vectors $\mathbf{0}$ simplifying (1.5):

$$^{\mathrm{i}}\mathbf{x} = \mathbf{R}_{\mathrm{ib}}\left({}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{b}}\mathbf{x}_{\mathrm{b}}\right) = \mathbf{R}_{\mathrm{ib}}\,{}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{i}}\mathbf{x}_{\mathrm{b}} \tag{1.6}$$

Multiple rotation matrices may be composed to represent successive rotations, multiplying from right to left. If $\mathbf{R}_{\mathrm{ib}}$ maps vectors from the body frame to the inertial frame, and $\mathbf{R}_{\mathrm{bs}}$ maps vectors from the sensor frame to the body frame, then the composite rotation matrix $\mathbf{R}_{\mathrm{is}}$ mapping vectors from the sensor frame to the inertial frame is given by:

$$\mathbf{R}_{\mathrm{is}} = \mathbf{R}_{\mathrm{ib}}\mathbf{R}_{\mathrm{bs}} \tag{1.7}$$

Rotation matrices are also known as DCMs because each element represents the cosine of the angle between the corresponding axes of the two coordinate frames.

$$\mathbf{R}_{\mathrm{ib}} = \begin{bmatrix} \hat{\mathbf{I}}_x \cdot \hat{\mathbf{B}}_x & \hat{\mathbf{I}}_x \cdot \hat{\mathbf{B}}_y & \hat{\mathbf{I}}_x \cdot \hat{\mathbf{B}}_z \\ \hat{\mathbf{I}}_y \cdot \hat{\mathbf{B}}_x & \hat{\mathbf{I}}_y \cdot \hat{\mathbf{B}}_y & \hat{\mathbf{I}}_y \cdot \hat{\mathbf{B}}_z \\ \hat{\mathbf{I}}_z \cdot \hat{\mathbf{B}}_x & \hat{\mathbf{I}}_z \cdot \hat{\mathbf{B}}_y & \hat{\mathbf{I}}_z \cdot \hat{\mathbf{B}}_z \end{bmatrix} \tag{1.8}$$

Because the dot product of two unit vectors $\hat{\mathbf{I}}$ and $\hat{\mathbf{B}}$ is:

$$\hat{\mathbf{I}}_i \cdot \hat{\mathbf{B}}_j = \|\hat{\mathbf{I}}_i\|\|\hat{\mathbf{B}}_j\| \cos\alpha_{ij} = \cos\alpha_{ij}, \quad i,j \in \{x,y,z\} \tag{1.9}$$

(1.8) may therefore be expressed in terms of the cosines of the angles between the corresponding axes. Let α_{ij} denote the angle between the inertial frame axis $\hat{\mathbf{I}}_i$ and the

body frame axis $\hat{\mathbf{B}}_j$, corresponding respectively to row i and column j of the rotation matrix (where $i, j \in \{x, y, z\}$).

$$\mathbf{R}_{\text{ib}} = \begin{bmatrix} \cos\alpha_{xx} & \cos\alpha_{xy} & \cos\alpha_{xz} \\ \cos\alpha_{yx} & \cos\alpha_{yy} & \cos\alpha_{yz} \\ \cos\alpha_{zx} & \cos\alpha_{zy} & \cos\alpha_{zz} \end{bmatrix} \tag{1.10}$$

example: The equation of the rotation shown in Fig. 1.1

$$\mathbf{R}_{\text{ib}} = \begin{bmatrix} \cos\alpha & \cos(\pi/2+\alpha) & \cos(\pi/2) \\ \cos(\pi/2-\alpha) & \cos\alpha & \cos(\pi/2) \\ \cos(\pi/2) & \cos(\pi/2) & \cos(0) \end{bmatrix} = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1.11}$$

We conclude with a brief discussion of the drawbacks and benefits of using rotation matrices for attitude representation.

Pros

- Direct Mapping: Rotation matrices provide a direct linear mapping of vectors via matrix multiplication.
- Singularity-Free: They are free from geometric singularities.
- Intuitive Kinematics: They provide a clear geometric interpretation of rotational kinematics whereby elements directly correspond to the relationship between frame unit vectors.
- Unique: Each unique 3-D rotation corresponds to exactly one matrix in SO(3).

Cons

- Redundant: They use nine elements to represent only three degrees of freedom.
- Orthogonality and Normalization: Maintaining the orthogonality and normalization of rotation matrices can be challenging due to numerical errors, often requiring re-canonization.
- Interpolation: Interpolating between rotation matrices is not straightforward and often requires conversion to other representations like quaternions for smooth interpolation.

Homogeneous Transformation Matrices

A common extension of the rotation matrix representation is the homogeneous transformation matrix [5], often used in robotics, which incorporates both rotation and translation components as described in (1.6). A homogeneous transformation matrix $\mathbf{T}_{\text{ib}}$ can be expressed as:

$$\mathbf{T}_{\text{ib}} = \begin{bmatrix} \mathbf{R}_{\text{ib}} & {}^{\text{i}}\mathbf{x}_{\text{b}} \\ \mathbf{0}_{1\times 3}^{\mathsf{T}} & 1 \end{bmatrix} \in \text{SE}(3), \tag{1.12}$$

where $\mathbf{R}_{\mathrm{ib}} \in \mathrm{SO}(3)$ and ${}^{\mathrm{i}}\mathbf{x}_{\mathrm{b}} \in \mathbb{R}^3$ is the position of the body frame origin relative to the inertial frame's origin, expressed in the inertial frame. The homogeneous transformation matrix $\mathbf{T}_{\mathrm{ib}}$ thus defines the *special Euclidean group* SE(3), which encompasses all possible rigid body transformations in three-dimensional space. Like rotation matrices, homogeneous transformation matrices may be multiplied from right to left to compose multiple transformations, allowing the transformation from one frame to another through intermediate frames.

The homogeneous transformation, along with the Denavit-Hartenberg (DH) parameters, provides a systematic way to describe the kinematics of robotic manipulators and the various transformations between different coordinate frames.

Quaternions

Quaternions, specifically *unit* quaternions are an efficient representation of rotations very commonly used in aerospace and robotics. They form the backbone of this work as it applies to optimization problems on SO(3). Trajectories and the corresponding attitude kinematics and dynamics for the spacecraft slew maneuvers are parameterized using unit quaternions. However, while the trajectories are parameterized using unit quaternions, rotation matrices are still used for most computations and transformations; as such, this section covers the mathematical properties and relations with respect to their application to this work. A more complete description of quaternions, their algebra, and their relationship to rotation matrices can be found in [5].

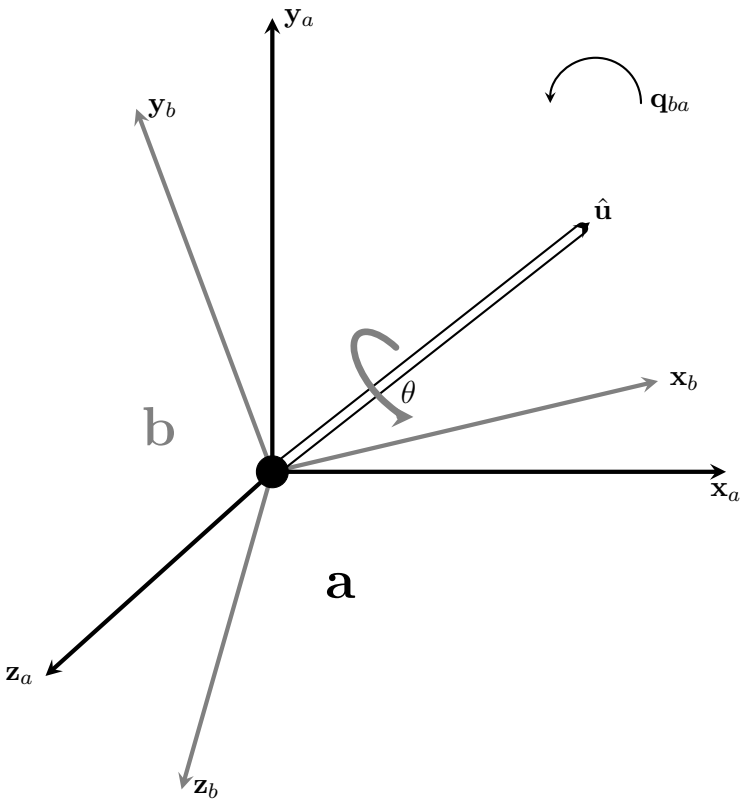


Figure 1.2: Initial frame a, rotated frame b, unit rotation axis $\hat{\mathbf{u}}$, rotation angle θ, and unit quaternion rotation $\mathbf{q}_{ba}$.

A quaternion, $\mathbf{q} \in \mathbb{H}$, consists of a scalar part q_s and a vector part $\mathbf{q_v} = [q_x, q_y, q_z]^\mathsf{T} \in \mathbb{R}^3$ which together form a four dimensional space (4-D) vector [5, 11] as depicted in Fig. 1.2. Two conventions exist: a scalar first convention and a scalar last convention; this work utilizes the scalar last convention:

$$\mathbf{q} = \begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} \mathbf{q_v} \\ q_s \end{bmatrix} = \begin{bmatrix} sin(\theta/2)\hat{\mathbf{u}} \\ cos(\theta/2) \end{bmatrix} \tag{1.13}$$

Generally, quaternions both rotate *and scale* vectors. However, a subset of quaternions, known as *unit quaternions*, only rotate vectors without scaling them. Unit quaternions satisfy:

$$\|\mathbf{q}\|_2 = \sqrt{q_x^2 + q_y^2 + q_z^2 + q_w^2} = 1. \tag{1.14}$$

The unit norm constraint is a critical property of our optimizations ensuring that the quaternions represent valid elements of SO(3).

Quaternion multiplication is used to represent relative rotations between orientations, where $\mathbf{q}_{\text{relative}}$ transforms the initial frame into the identity orientation, represented by the identity quaternion $\mathbf{q}_{\text{identity}} = [\mathbf{0}_{3\times1}^\mathsf{T}, 1]^\mathsf{T}$.

$$\overline{\mathbf{q}} = \begin{bmatrix} -\mathbf{q_v} \\ q_s \end{bmatrix}, \tag{1.15}$$

$$\mathbf{q}^{-1} = \frac{\overline{\mathbf{q}}}{\|\mathbf{q}\|_2^2}. \tag{1.16}$$

For unit quaternions the inverse equals the conjugate, i.e., $\mathbf{q}^{-1} = \overline{\mathbf{q}}$. The relative rotation quaternion is then:

$$\mathbf{q}_{\text{relative}} = \mathbf{q}_{\text{initial}}^{-1} \otimes \mathbf{q}_{\text{final}}, \tag{1.17}$$

$$\mathbf{q}_1 \otimes \mathbf{q}_2 = \begin{bmatrix} q_{s,1}\mathbf{q}_{v,2} + q_{s,2}\mathbf{q}_{v,1} + \mathbf{q}_{v,1} \times \mathbf{q}_{v,2} \\ q_{s,1}q_{s,2} - \mathbf{q}_{v,1} \cdot \mathbf{q}_{v,2} \end{bmatrix} \tag{1.18}$$

A relative quaternion, $\mathbf{q}_{\text{relative}}$, represents the quaternion that rotates from the initial orientation to the final orientation and is useful for the mapping of MRPs to SO(3).

Quaternions have a property known as the *double-cover*:

$$\mathbf{R}(\mathbf{q}) = \mathbf{R}(-\mathbf{q}) \in \text{SO}(3), \tag{1.19}$$

meaning that both represent the same rotation. To avoid ambiguity in representing rotations, it is common practice to enforce a convention, such as requiring the scalar part of the quaternion to be non-negative, i.e., $q_s \geq 0$.

$$\mathbf{q} \to \begin{cases} \mathbf{q}, & q_s \geq 0 \\ -\mathbf{q}, & q_s < 0 \end{cases} \tag{1.20}$$

Enforcing this convention ensures a unique representation for each rotation and obviates an optimized slew taking the longer path due to quaternion sign ambiguity. Along with the unit norm constraints and the scalar last convention, quaternion negation functions as the final condition of canonical quaternion representation for this work. Enforcing these conditions is known as quaternion canonization.

A final utility function is the conversion from quaternions to rotation matrices:

$$R_{\text{bi}}(q) = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_x q_y + q_w q_z) & 2(q_x q_z - q_w q_y) \\ 2(q_x q_y - q_w q_z) & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_y q_z + q_w q_x) \\ 2(q_x q_z + q_w q_y) & 2(q_y q_z - q_w q_x) & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}. \tag{1.21}$$

Like rotation matrices, quaternions have their own set of pros and cons:

Pros

- Singularity-free: Avoids the inherent singularities present in all smooth minimal attitude representations, allowing for continuous and well-defined attitude representation on the entirety of SO(3).
- Computationally Efficient: Contain only four elements—far fewer parameters than rotation matrices—and rely purely upon algebraic operations for frame transformations, avoiding transcendental trigonometric functions.
- Smooth Interpolation: Allow for smooth interpolation between rotations using Spherical Linear Interpolation (SLERP).

Cons

- Unit Norm Constraint: Only unit quaternions represent valid rotations, necessitating normalization after numerical operations to maintain the unit norm. The unit norm constraint also poses challenges in optimization frameworks as the feasible set exists on a 3-D manifold in 4-D space.
- Double Cover: q and $-q$ map to the same physical rotation, introducing ambiguity that must be managed via consistent canonization, especially in optimization algorithms to avoid taking longer paths around the sphere.
- Redundant: They use four elements versus the three element minimal representation.
- Geometrically Confusing: Being four-dimensional hyperspheres, quaternions are not easily conceptualized in comparison to other attitude representations.

MRPs

MRPs represent the final attitude representation discussed in this work, offering a three-parameter minimal parameterization of rotations derived as stereographic projections of the unit quaternion hypersphere onto a 3-D hyperplane [11].

MRPs were introduced in this work to implement the Inverse-dynamics Particle Swarm Optimization (IPSO) algorithm—which due to characteristics of how Particle Swarm Optimization (PSO) algorithms explore search spaces, made quaternions, which require the unit norm constraint, impractical for this algorithm class.

This representation is defined in terms of the rotation axis $\hat{\mathbf{u}}$ and the rotation angle θ [11] as shown in Fig. 1.2:

$$\mathbf{p}(\theta, \hat{\mathbf{u}}) = \tan\left(\frac{\theta}{4}\right)\hat{\mathbf{u}} = \frac{\mathbf{q}_v}{1 + q_w}, \tag{1.22}$$

As a minimal representation, MRPs avoid redundancy inherent to quaternions and rotation matrices, but succumb unavoidably to either singularities or discontinuities. From (1.22), it is evident that MRPs become singular when the rotation angle θ approaches 2π, as $\tan\left(\frac{\theta}{4}\right)$ approaches infinity. If the angle of rotation is bounded less than 2π, MRPs provide a continuous, singularity-free representation of rotations; however, spacecraft maneuvers are not constrained as such and singularities may appear [11]. One common approach to mitigate this issue is to switch to the shadow set of MRPs when the norm of the current set exceeds unity—occurring for rotations greater than π—effectively avoiding the singularity at 2π, but this introduces instead a discontinuity that optimizers struggle to handle.

Conversion between MRPs and quaternions or rotation matrices occur via:

$$\mathbf{q}(\mathbf{p}) = \frac{1}{1 + p^2}\begin{bmatrix} 2\mathbf{p} \\ 1 - p^2 \end{bmatrix} \tag{1.23}$$

$$\mathbf{R}_{bi}(\mathbf{p}) = I_{3\times 3} - \frac{4(1 - p^2)}{(1 + p^2)^2}[\mathbf{p}]_\times + \frac{8}{(1 + p^2)^2}[\mathbf{p}]_\times^2 \tag{1.24}$$

Naturally, MRPs come with their own set of tradeoffs:

Pros

- Minimal Representation: Uses exactly three parameters to describe three DOF rotations, eliminating the unit-norm constraint required by quaternions.
- Extended Singularity: As opposed to classic Rodrigues parameters, MRPs exhibit singularities only at rotations of 2π rather than π, providing a larger singularity-free region for practical rotations.
- Transcendental-free Kinematics: Unlike Euler angles, the kinematic differential equations for MRPs do not involve trigonometric functions.

Cons

- Singular or Discontinuous: Avoiding singularities involves switching to the shadow set for rotations greater than π, which can introduce discontinuities; without the shadow set, the representation is smooth but contains singularities.

- Spatial Warping: Due to MRPs being a *stereographic projection* of a 4-D hypersphere onto a 3-D hyperplane, straight paths in the MRP space *generally* do not correspond geodesics (great circles) on SO(3); a straight line trajectory only yields a true geodesic if the trajectory passes through the origin ($\mathbf{p} = \mathbf{0}$) and typically requires the use of relative rotations between coordinate frames to help align maneuvers with the origin. The distortion induces a change in the loss landscape, which affects optimization and control strategies.

1.1.2 Manipulator Kinematics

To describe the kinematics of the 7-DOF robotic arm, the Modified Denavit-Hartenberg (MDH) parameter convention is employed to relate consecutive link coordinate frames and systematically generate homogeneous transformation matrices in (1.12). The MDH convention was introduced by John J. Craig [4], and is a slight reformation of the standard DH convention, where the coordinate frames are attached differently to the links, primarily in how the parameters are defined relative to the link origins. MDH uses four parameters to describe the 6-DOF pose between link frames.

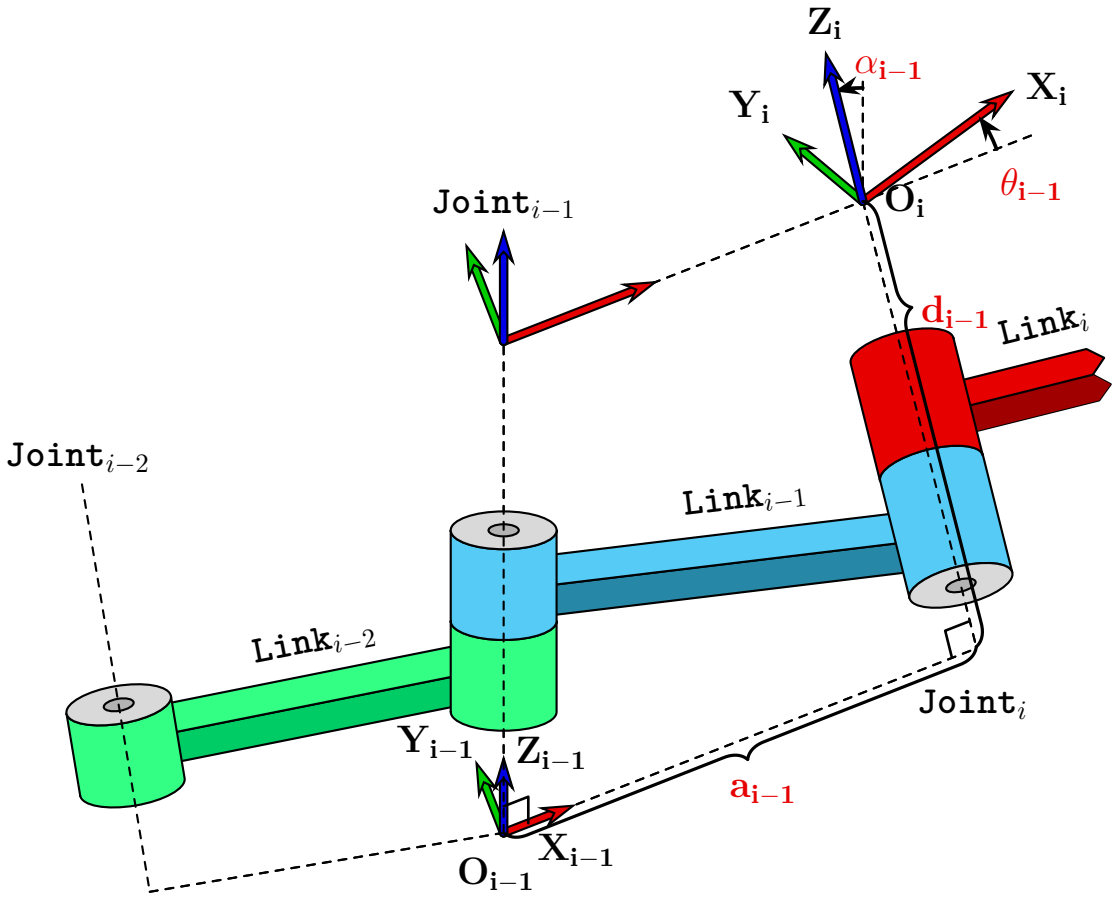


Figure 1.3: Modified Denavit–Hartenberg (MDH) parameters for consecutive link frames.

Modified Denavit–Hartenberg Parameters

Unlike the standard DH convention which attaches frame i to the end-effector side—where link i connects to link $i+1$—the modified convention attaches frame i to the base side—where link i connects to link $i-1$—as shown in Fig. 1.3. The mapping of frame i to frame $i-1$ is characterized by four geometric parameters [4]:

- α_{i-1}: **Link twist**, the angle from $\hat{\mathbf{Z}}_{i-1}$ to $\hat{\mathbf{Z}}_i$ about the $\hat{\mathbf{X}}_{i-1}$ axis.
- a_{i-1}: **Link length**, the distance from $\hat{\mathbf{Z}}_{i-1}$ to $\hat{\mathbf{Z}}_i$ along the $\hat{\mathbf{X}}_{i-1}$ axis.
- θ_{i-1}: **Joint angle**, the angle from $\hat{\mathbf{X}}_{i-1}$ to $\hat{\mathbf{X}}_i$ about the $\hat{\mathbf{Z}}_i$ axis.
- d_{i-1}: **Link offset**, the distance from $\hat{\mathbf{X}}_{i-1}$ to $\hat{\mathbf{X}}_i$ along the $\hat{\mathbf{Z}}_i$ axis.

For a revolute joint, θ_{i-1} is the variable DOF, while a_{i-1}, α_{i-1}, and d_{i-1} are constants unique to the link geometry. The homogeneous transformation matrix $\mathbf{T}_{i,i-1} \in \mathrm{SE}(3)$ connecting to frame i from frame $i-1$ is constructed from the transformation sequence: α_{i-1}, a_{i-1}, θ_{i-1}, and d_{i-1}. Multiplying these sequential transformations yields the homogeneous transformation matrix:

$$\mathbf{T}_{i,i-1}(\theta_{i-1}) = \begin{bmatrix} cos\theta_{i-1} & -sin\theta_{i-1} & 0 & a_{i-1} \\ sin\theta_{i-1}cos\alpha_{i-1} & cos\theta_{i-1}cos\alpha_{i-1} & -sin\alpha_{i-1} & -sin\alpha_{i-1}d_{i-1} \\ sin\theta_{i-1}sin\alpha_{i-1} & cos\theta_{i-1}sin\alpha_{i-1} & cos\alpha_{i-1} & cos\alpha_{i-1}d_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.25}$$

By defining these four parameters for each of the seven joints of the manipulator, the forward kinematics may be computed. As described in § 1.1.1, homogeneous transformation matrices can be multiplied to represent successive coordinate changes. In this manner, the pose of the end-effector (frame 7) relative to the manipulator base (frame 0) is obtained by chaining successive transformations:

$$\mathbf{T}_{0,7} = \prod_{i=1}^{7} \mathbf{T}_{i-1,i} = \mathbf{T}_{0,1}\mathbf{T}_{1,2}\cdots\mathbf{T}_{6,7} \tag{1.26}$$

This transformation matrix $\mathbf{T}_{0,7} \in \mathrm{SE}(3)$ yields the complete 6-D pose of the end-effector. Within the scope of this work, generating these transformation matrices forms the foundation of the manipulator's kinematic model.

1.2 B-Splines

To transcribe the continuous-time optimal control problem (OCP) into a discretized time, finite-dimensional nonlinear program (NLP) solvable by algorithms such as Interior Point OPTimizer (IPOPT), the state and control trajectories are parameterized using B-splines. B-splines (Basis splines) are piecewise polynomial functions defined over a sequence of values known as a knot vector—with each element greater than or equal to

the preceding one. This parameterization defines the trajectory's shape as a discrete set of control vertices, reducing the infinite-dimensional optimization to a finite set of decision variables while preserving continuous kinematics [3]. A B-spline curve $\mathbf{x}(s)$ of degree p is constructed from n control vertices $\mathbf{c}_i \in \mathbb{R}^d (i = 0, \ldots, n-1)$ and corresponding scalar basis functions $N_{i,p}(s)$:

$$\mathbf{x}(s) = \sum_{i=0}^{n-1} \mathbf{c}_i N_{i,p}(s), \tag{1.27}$$

where $s \in [0, 1]$ is the normalized time parameter along the trajectory. The basis functions $N_{i,p}(s)$ are computed recursively over a knot vector $\mathbf{T} = \{t_0, t_1, \ldots, t_{n+p+1}\}$ via the Cox–de Boor algorithm [3]:

$$N_{i,0}(s) = \begin{cases} 1 & \text{if } t_i \leq s < t_{i+1} \\ 0 & \text{otherwise} \end{cases}, \tag{1.28}$$

$$N_{i,p}(s) = \frac{s - t_i}{t_{i+p} - t_i} N_{i,p-1}(s) + \frac{t_{i+p+1} - s}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(s). \tag{1.29}$$

In this formulation, $0/0 \equiv 0$ explicitly to handle identical sequential knots.

Knot Vector and Clamping

This work employs *clamped, uniform* knot vectors. A clamped knot vector duplicates the first and last knots $p + 1$ times which forces the curve to exactly start and end at the first and last control vertices: $\mathbf{x}(0) = \mathbf{c}_0$ and $\mathbf{x}(1) = \mathbf{c}_{n-1}$. The clamped structure ensures the trajectory deterministically obeys the boundary conditions of the NLP—such as the initial and final orientation of a spacecraft slew, or the terminal intercept position of a robotic manipulator—through the corresponding boundary control vertices. The uniform property means the internal knots are uniformly distributed—with equal knot spans between consecutive knots—simplifying the evaluation time grid.

Properties Critical for Trajectory Optimization

The B-spline parameterization provides three mathematical properties essential for formulating kinodynamic optimization problems:

- **Convex Hull Property:** A B-spline curve is strictly contained within the convex hull of the control vertices defining any given segment—i.e. the values of the trajectory are bounded by the values of the influencing control vertices. Consequently, if the control vertices are bounded by a maximum magnitude, the resulting continuous trajectory is strictly bound by the same magnitude—no overshoot occurs, unlike many other trajectory via spline interpolation methods. This enables the NLP solver to enforce continuous-time state bounds (e.g., joint limits) by applying simple bound constraints directly to the discrete control vertices. The convex hull property also applies to the derivative B-spline trajectories.
- **Local Support:** The basis function $N_{i,p}(s)$ is non-zero only within the interval $[t_i, t_{i+p+1})$. Modifying a single control vertex $\mathbf{c}_i$ alters the trajectory only within

this local span. In the context of the NLP, local support yields sparse and banded Jacobian and Hessian matrices, which interior-point solvers exploit for efficient, large-scale matrix calculation. Furthermore, this property prevents individual control vertex updates from inducing global trajectory oscillations, which is a common drawback of global polynomial parameterizations.

- **Analytic Differentiation:** The time derivative of a degree p B-spline is analytically defined as another B-spline of degree $p-1$. The derivative trajectory is parameterized by a new set of control vertices $\mathbf{v}_i$, derived from the finite differences of the position vertices:

$$\dot{\mathbf{x}}(s) = \sum_{i=0}^{n-2} \mathbf{v}_i N_{i+1,p-1}(s), \tag{1.30}$$

where the velocity control vertices are defined as:

$$\mathbf{v}_i = \frac{\mathrm{d}}{\mathrm{d}s} N_{i+1,p-1}(s) = p \frac{\mathbf{c}_{i+1} - \mathbf{c}_i}{t_{i+p+1} - t_{i+1}}. \tag{1.31}$$

All higher-order derivatives (acceleration, jerk, etc.) follow the same pattern, eliminating the need for finite-difference approximations or numerical integration when evaluating the forward and inverse dynamics of the spacecraft and robotic manipulator models.

B-Spline Trajectory Time Scaling

The normalized spline parameter $s \in [0,1]$ maps to physical time $t \in [t_0, t_f]$ via $s(t) = \frac{t-t_0}{T}$, where $T = t_f - t_0$ represents the total trajectory duration. The physical trajectory $\mathbf{x}(t)$ is evaluated from the normalized parameterization as:

$$\mathbf{x}(t) = \mathbf{x}(s(t)) = \sum_{i=0}^{n-1} \mathbf{c}_i N_{i,p}\left(\frac{t-t_0}{T}\right). \tag{1.32}$$

Applying the chain rule yields the physical time derivatives required for evaluating system dynamics and kinematic limits:

$$\frac{\mathrm{d}^k \mathbf{x}(t)}{\mathrm{d}t^k} = \frac{1}{T^k} \frac{\mathrm{d}^k \mathbf{x}(s)}{\mathrm{d}s^k}, \quad \text{for } k \geq 0. \tag{1.33}$$

Consequently, the physical velocity $\dot{\mathbf{x}}(t)$ and acceleration $\ddot{\mathbf{x}}(t)$ are given by scaling the parameter derivatives by $\frac{1}{T^k}$. If T is optimized as a decision variable in a free-final-time OCP, this scaling directly influences the path constraints and required control torques.

1.3 Physical Models, Kinematics, Dynamics, & Constraints

This thesis establishes two physical models:

- Spacecraft attitude dynamic model for pure rotational motion.
- 7-DOF robotic manipulator dynamic model.

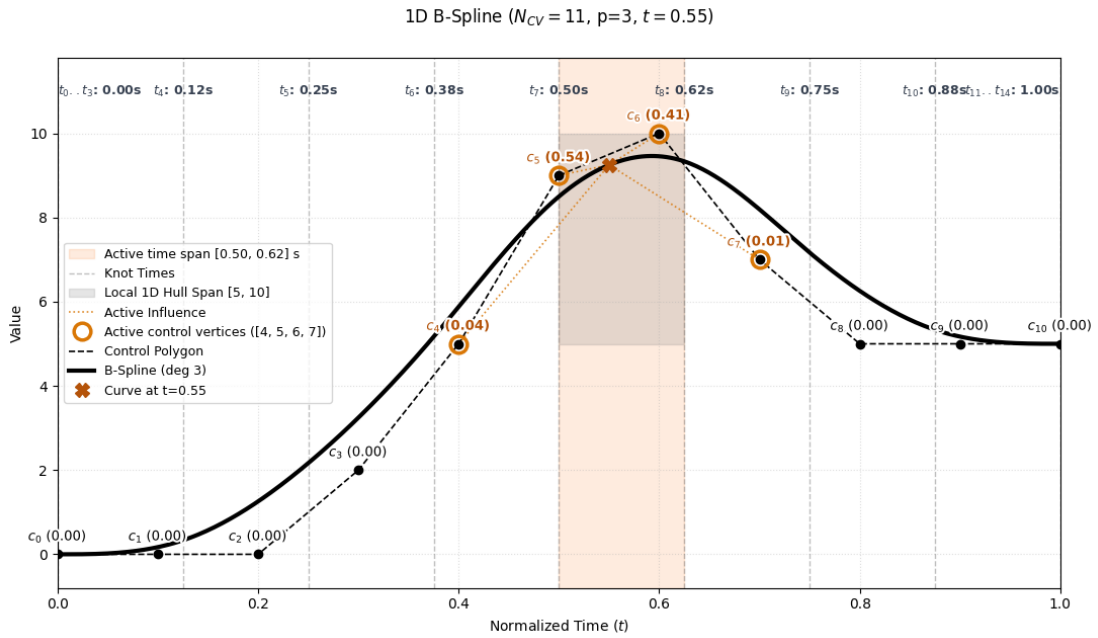


Figure 1.4: 1-D cubic B-spline trajectory demonstrating the local support property and control vertex influence at an example point $t = 0.55\,\text{s}$. The active knot span $[t_7, t_8] = [0.50, 0.62]\,\text{s}$ dictates that the curve value is determined by control vertices $\mathbf{c}_4$ through $\mathbf{c}_7$, with basis weights $N_{i,3}$ corresponding to active basis functions. Basis weights vary depending on the value of the basis functions for the evaluated point t, respective only to its location within the active knot span. Clamping the knot vector ($\{t_0, \ldots, t_3\}$ and $\{t_{11}, \ldots, t_{14}\}$) anchors the curve endpoints to $\mathbf{c}_0$ and $\mathbf{c}_{10}$, while repeating the initial and final three control vertex values enforces zero derivatives up to second order ($\dot{\mathbf{q}} = \mathbf{0}, \ddot{\mathbf{q}} = \mathbf{0}$) at the boundaries. The grey shaded region indicates local convex hull bounds $[5, 10]$.

The function of these models is to serve as the foundational physical plants for the trajectory optimization algorithms evaluated in this thesis. Both physical models are formulated under ideal rigid-body assumptions. Consequently, higher-order dynamic effects—including structural flexibility, fluid slosh, actuator and joint non-idealities, and environmental disturbances such as gravity gradient torques, solar radiation pressure, and drag forces are omitted from the system equations.

Our implementation uses a direct B-spline inverse-dynamic trajectory parameterization for both models, whereby the continuous-time state trajectories are evaluated at discrete sampling times for enforcing control and kinodynamic constraints, eliminating the need for numerical integration of the equations of motion (EOM).

This section introduces both kindodynamic plant models and their associated constraints.

1.3.1 Spacecraft Model

The spacecraft model describes pure rotational motion—existing on SO(3)—ignoring any translational motion—which would be encompassed by special Euclidean group in three dimensions (SE(3)). The optimization parameters are purely the B-spline control vertices for the quaternion trajectory, $\mathbf{q}$,with all other kinematic and dynamic quantities derived from the quaternions and their derivatives.

Quaternion Trajectories and Their Derivatives

If the quaternion control vertex matrix is $\mathbf{P} \in \mathbb{R}^{4\times n_{ctrl}}$ and the B-spline basis matrix is $\boldsymbol{\varphi} \in \mathbb{R}^{n_{ctrl}\times 1}$:

$$\mathbf{q} = \mathbf{P}\boldsymbol{\varphi} \in \mathbb{R}^{4\times 1}, \tag{1.34}$$

or for the entire trajectory over all sampling times, we can write:

$$\mathbf{Q} = \mathbf{P}\boldsymbol{\Phi} \in \mathbb{R}^{4\times n_{via}}, \tag{1.35}$$

where $\mathbf{Q}$ is the matrix of quaternion values at all sampling times and $\boldsymbol{\Phi} \in \mathbb{R}^{n_{ctrl}\times n_{via}}$ is the B-spline basis matrix evaluated at all sampling times. n_{ctrl} is the number of control vertices—equivalently the number of elements in the optimization vector—parameterizing the B-spline and n_{via} represents the number of via points or sampling times for the trajectory.

$$\dot{\mathbf{q}} = \mathbf{P}\dot{\boldsymbol{\varphi}} \in \mathbb{R}^{4\times 1} \tag{1.36}$$
$$\dot{\mathbf{Q}} = \mathbf{P}\dot{\boldsymbol{\Phi}} \in \mathbb{R}^{4\times n_{\text{via}}} \tag{1.37}$$

$$\ddot{\mathbf{q}} = \mathbf{P}\ddot{\boldsymbol{\varphi}} \in \mathbb{R}^{4\times 1} \tag{1.38}$$
$$\ddot{\mathbf{Q}} = \mathbf{P}\ddot{\boldsymbol{\Phi}} \in \mathbb{R}^{4\times n_{\text{via}}} \tag{1.39}$$

$$\dddot{\mathbf{q}} = \mathbf{P}\dddot{\boldsymbol{\varphi}} \in \mathbb{R}^{4\times 1} \tag{1.40}$$
$$\dddot{\mathbf{Q}} = \mathbf{P}\dddot{\boldsymbol{\Phi}} \in \mathbb{R}^{4\times n_{\text{via}}} \tag{1.41}$$

maybe include the EOMs for the MRP formulation???

Equations of Motion (EOM)

The state vector of the spacecraft is defined as $\mathbf{x} = [\mathbf{q}^\top, \boldsymbol{\omega}^\top]^\top$, where $\mathbf{q} \in S^3 \subset \mathbb{H}$ represents the attitude quaternion and $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity vector. The rotational dynamics of the spacecraft are governed by Euler's equation:

$$\boldsymbol{\tau} = \mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}), \tag{1.42}$$

where $\mathbf{J}$ is the spacecraft's inertia matrix, $\boldsymbol{\omega}$ is the angular velocity vector, and τ represents the control torque *input* applied by the actuators.

The relation between the quaternion rate and the angular velocity is:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega} = \frac{1}{2}\mathbf{C}(\mathbf{q})^\top \boldsymbol{\omega}, \tag{1.43}$$

$$2\mathbf{C}(\mathbf{q})\dot{\mathbf{q}} = \boldsymbol{\omega}, \tag{1.44}$$

$$\mathbf{C}(\mathbf{q}) = \begin{bmatrix} q_w \mathbf{I}_{3\times 3} + [\mathbf{q_v}]_\times & -\mathbf{q_v} \end{bmatrix} = \begin{bmatrix} q_w & q_z & -q_y & -q_x \\ -q_z & q_w & q_x & -q_y \\ q_y & -q_x & q_w & -q_z \end{bmatrix} \tag{1.45}$$

$$\mathbf{C}(\mathbf{q})\mathbf{C}(\mathbf{q})^\top = \mathbf{I}_{3\times 3}, \tag{1.46}$$

where $\mathbf{C}(\mathbf{q})$ is the quaternion differential matrix.

Applying the chain rule, the time derivative of the angular velocity $\dot{\boldsymbol{\omega}}$—i.e. angular acceleration—and the second time derivative $\ddot{\boldsymbol{\omega}}$—i.e. jerk—can be expressed in terms of the quaternion and its derivatives:

$$\dot{\boldsymbol{\omega}} = 2\left[\mathbf{C}(\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{C}(\mathbf{q})\ddot{\mathbf{q}}\right]. \tag{1.47}$$

$$\ddot{\boldsymbol{\omega}} = 2\left[\mathbf{C}(\ddot{\mathbf{q}})\dot{\mathbf{q}} + 2\mathbf{C}(\dot{\mathbf{q}})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q})\,\dddot{\mathbf{q}}\right]. \tag{1.48}$$

Alternatively, if the angular velocity, angular acceleration, or jerk are known—as is the case for enforcing boundary constraints on the OCP—one can solve for the derivatives of the quaternion as:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{C}(\mathbf{q})^\top \boldsymbol{\omega}, \tag{1.49}$$

$$\ddot{\mathbf{q}} = \frac{1}{2}\left[\mathbf{C}(\mathbf{q})^\top \dot{\boldsymbol{\omega}} + \mathbf{C}(\dot{\mathbf{q}})^\top \boldsymbol{\omega}\right], \tag{1.50}$$

$$\dddot{\mathbf{q}} = \frac{1}{2}\left[\mathbf{C}(\mathbf{q})^\top \ddot{\boldsymbol{\omega}} + 2\mathbf{C}(\dot{\mathbf{q}})^\top \dot{\boldsymbol{\omega}} + \mathbf{C}(\ddot{\mathbf{q}})^\top \boldsymbol{\omega}\right]. \tag{1.51}$$

For the interior of the OCP, where the optimizer has freedom to optimize interior control vertices, $\mathbf{c}_{int}$, and therefore the trajectory, the order of operations follows: $\boxed{\mathbf{c}_{int} \longrightarrow \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}} \longrightarrow \boldsymbol{\omega}, \dot{\boldsymbol{\omega}} \longrightarrow \boldsymbol{\tau}}$.

While at the initial and final boundaries, where the quaternion and its derivatives are constrained by equality constraints, the exterior control vertices $\mathbf{c}_{ext}$ are determined by: $\boxed{\mathbf{q}, \boldsymbol{\omega}, \dot{\boldsymbol{\omega}}, \ddot{\boldsymbol{\omega}} \longrightarrow \dot{\mathbf{q}}, \ddot{\mathbf{q}}, \dddot{\mathbf{q}} \longrightarrow \mathbf{c}_{ext}}$.

Exterior Control Vertex Calculation

As stated in the previous section, the exterior control vertices are determined by the boundary conditions on the quaternion trajectory and its derivatives. This section details the procedure for analytically and deterministically calculating the outermost control vertices of the B-spline trajectory. This reduces both the number of optimization variables and the computational complexity of the OCP as boundary condition kinematics are directly solved rather than passed to the optimizer to be enforced via equality constraints.

The quaternion B-spline control-vertex matrix is partitioned into the boundary and interior control vertices:

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_0 & \mathbf{P}_{\text{int}} & \mathbf{P}_f \end{bmatrix} \in \mathbb{R}^{4\times n_{ctrl}}, \tag{1.52}$$

where $\mathbf{P}_0$ contains the first n_{BC} control vertices, $\mathbf{P}_f$ contains the last n_{BC} control vertices, and $\mathbf{P}_{\text{int}}$ contains the interior free control vertices.

Consequently,

$$n_{ctrl} = n_{\text{FCV}} + 2n_{\text{BC}}, \tag{1.53}$$

where n_{FCV} is the number of free control vertices. After the exterior vertices are determined from the boundary conditions, only $\mathbf{P}_{\text{int}}$ is treated as an optimization variable.

Let $\boldsymbol{\varphi}^{(r)}(t)$ denote the rth derivative of the B-spline basis vector at time t. The boundary basis matrices are formed from the active control-vertex entries at the initial and final times:

$$\mathbf{B}_0 = \begin{bmatrix} \boldsymbol{\varphi}_0^{(0)} & \boldsymbol{\varphi}_0^{(1)} & \boldsymbol{\varphi}_0^{(2)} \end{bmatrix} \in \mathbb{R}^{n_{\text{BC}}\times n_{\text{BC}}}, \tag{1.54}$$

$$\mathbf{B}_f = \begin{bmatrix} \boldsymbol{\varphi}_f^{(0)} & \boldsymbol{\varphi}_f^{(1)} & \boldsymbol{\varphi}_f^{(2)} \end{bmatrix} \in \mathbb{R}^{n_{\text{BC}}\times n_{\text{BC}}}, \tag{1.55}$$

where

$$\boldsymbol{\varphi}_0^{(r)} = \boldsymbol{\varphi}^{(r)}(t_0) \tag{1.56}$$

contains the first n_{BC} entries, and

$$\boldsymbol{\varphi}_f^{(r)} = \boldsymbol{\varphi}^{(r)}(t_f) \tag{1.57}$$

contains the last n_{BC} entries.

For the default case, $n_{\text{BC}} = 3$, the prescribed boundary conditions are quaternion position, quaternion rate, and quaternion acceleration:

$$\mathbf{Y}_0 = \begin{bmatrix} \mathbf{q}_0 & \dot{\mathbf{q}}_0 & \ddot{\mathbf{q}}_0 \end{bmatrix} \in \mathbb{R}^{4\times 3}, \tag{1.58}$$

$$\mathbf{Y}_f = \begin{bmatrix} \mathbf{q}_f & \dot{\mathbf{q}}_f & \ddot{\mathbf{q}}_f \end{bmatrix} \in \mathbb{R}^{4\times 3}. \tag{1.59}$$

Using

$$\mathbf{q}^{(r)}(t) = \mathbf{P}\boldsymbol{\varphi}^{(r)}(t), \tag{1.60}$$

the boundary conditions can be written as

$$\mathbf{P}_0\mathbf{B}_0 = \mathbf{Y}_0, \qquad \mathbf{P}_f\mathbf{B}_f = \mathbf{Y}_f. \tag{1.61}$$

The exterior control vertices are therefore obtained analytically as

$$\mathbf{P}_0 = \mathbf{Y}_0\mathbf{B}_0^{-1}, \qquad \mathbf{P}_f = \mathbf{Y}_f\mathbf{B}_f^{-1}. \tag{1.62}$$

Invertibility note. For a clamped B-spline, the zeroth-order basis function at $t = 0$ depends only on the first control vertex. The first derivative introduces the second control vertex, the second derivative introduces the third control vertex, and so on. Therefore, for $N_{\mathrm{BC}} = 3$,

$$\mathbf{B}_0 = \begin{bmatrix} \boldsymbol{\varphi}_0^{(0)} & \boldsymbol{\varphi}_0^{(1)} & \boldsymbol{\varphi}_0^{(2)} \end{bmatrix} \tag{1.63}$$

has an upper-triangular structure

$$\mathbf{B}_0 \sim \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \end{bmatrix}. \tag{1.64}$$

The final endpoint matrix has an analogous lower-triangular structure:

$$\mathbf{B}_f \sim \begin{bmatrix} \star & 0 & 0 \\ \star & \star & 0 \\ \star & \star & \star \end{bmatrix}. \tag{1.65}$$

The diagonal entries are nonzero if

$$\text{degree} \geq N_{\mathrm{BC}} - 1, \tag{1.66}$$

Hence,

$$\det(\mathbf{B}_0) \neq 0, \qquad \det(\mathbf{B}_f) \neq 0, \tag{1.67}$$

guaranteeing that $\mathbf{B}_0$ and $\mathbf{B}_f$ are invertible and (1.62) exists and is unique.

The quaternion derivatives appearing in $\mathbf{Y}_0$ and $\mathbf{Y}_f$ are obtained from the angular quantities using (1.49)–(1.51).

If $n_{\mathrm{BC}} = 4$, jerk is also imposed at both boundaries. In that case,

$$\mathbf{Y}_0 = \begin{bmatrix} \mathbf{q}_0 & \dot{\mathbf{q}}_0 & \ddot{\mathbf{q}}_0 & \dddot{\mathbf{q}}_0 \end{bmatrix}, \tag{1.68}$$

$$\mathbf{Y}_f = \begin{bmatrix} \mathbf{q}_f & \dot{\mathbf{q}}_f & \ddot{\mathbf{q}}_f & \dddot{\mathbf{q}}_f \end{bmatrix}, \tag{1.69}$$

and the boundary matrices become

$$\mathbf{B}_0 = \begin{bmatrix} \boldsymbol{\varphi}_0^{(0)} & \boldsymbol{\varphi}_0^{(1)} & \boldsymbol{\varphi}_0^{(2)} & \boldsymbol{\varphi}_0^{(3)} \end{bmatrix}, \tag{1.70}$$

$$\mathbf{B}_f = \begin{bmatrix} \boldsymbol{\varphi}_f^{(0)} & \boldsymbol{\varphi}_f^{(1)} & \boldsymbol{\varphi}_f^{(2)} & \boldsymbol{\varphi}_f^{(3)} \end{bmatrix}. \tag{1.71}$$

Therefore, the exterior control vertices are fixed deterministically from the boundary conditions, while the optimizer searches only over $\mathbf{P}_{\mathrm{int}}$.

State and Control Vertex Bounds

A helpful property of parameterizing SO(3) via unit quaternions $\mathbf{q} \in S^3 \subset \mathbb{R}^4$ is that S^3 is a compact manifold, meaning its search space in the Euclidean $\mathbb{R}^4$ is both bounded and closed.

Definition 1.1 (Bounded)**.** A set $\mathcal{S} \subset \mathbb{R}^n$ of a Euclidean space is said to be *bounded* if there exists a finite upper bound $M > 0$ on the magnitude of all its vectors: $|\mathbf{x}|_2 \leq M, \forall \mathbf{x} \in \mathcal{S}$.
Note: for unit quaternions, the maximum value of an element is one, making it bounded with $M = 1$. ▲

Definition 1.2 (Closed)**.** A set $\mathcal{S}$ of a metric space $(\mathcal{M}, d)$ is said to be *closed* if it contains all its limit points. That is, if a sequence $\{x_n\} \subset \mathcal{S}$ converges to a point $x \in \mathcal{M}$ (e.g., $d(x_n, x) \to 0$ as $n \to \infty$), then x must also belong to $\mathcal{S}$ ($x \in \mathcal{S}$). ▲

Definition 1.3 (Compact)**.** In Euclidean space $\mathbb{R}^n$, a subset $\mathcal{S}$ is *compact* if and only if it is both *bounded* and *closed* (Heine-Borel theorem). ▲

On S^3, all quaternion elements q_i belong to the compact set $[-1, 1]/\|\mathbf{q}\|_2 = 1$, preventing the state space from extending to infinity. While individual B-spline control vertices $\mathbf{P}$ parameterizing $\mathbf{q}$ may extend beyond $[-1, 1]^4$, the unit norm enforcement at each via point keeps the control vertex polygon tightly constrained near the unit quaternion bounds.

The only remaining concern involves the initialization of the problem, as we have not defined a concrete bound for the control vertices. Two clear strategies were considered:

1. Initialize the control vertices within the unit hypercube $[-1, 1]^4$, ensuring that they are close to the unit quaternion manifold—which results in a set of quaternions near but not on S^3—and allow the optimizer to refine them during the optimization process.
2. Generate an initial set of quaternions on S^3 and use them to calculate the initial control vertices, ensuring that the trajectory starts exactly on the unit quaternion manifold.

To the author's surprise, the first strategy resulted in a faster wall-clock computation time. The performance gain is likely due to the elimination of the preliminary matrix inversion required by the latter strategy during high-level Python orchestration, instead allowing the C++-based optimizer core to pull the trajectory onto the manifold smoothly during early iterations. The properties of S^3 alongside either initialization strategy allow us to easily condition the search space of the optimizer.

Sampling on SO(3)

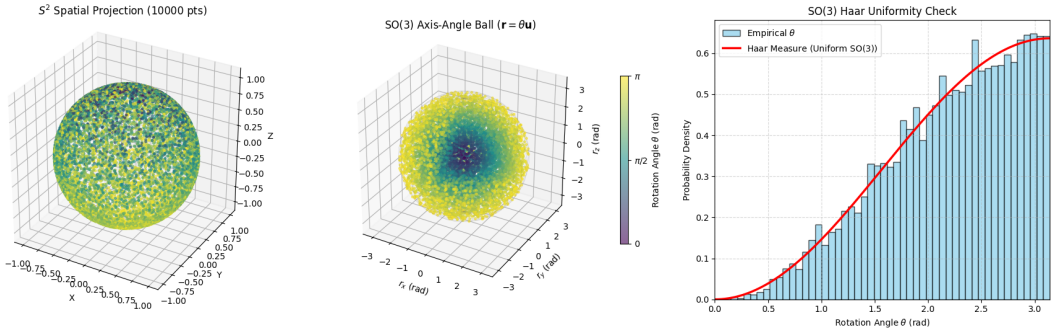
When searching the Special Orthogonal group $SO(3)$, sampling orientations requires careful consideration to ensure that the sample distribution corresponds to the Haar measure—the unique bi-invariant volume measure on the rotation group [6, 8]. A naïve approach to quaternion sampling consists of independently drawing four scalar components $w, x, y, z \sim \mathcal{U}[-1, 1]$ in $\mathbb{R}^4$ and normalizing the resulting vector to unit length. This

approach fails to produce a uniform distribution on S^3; the geometry of the bounding 4D hypercube $[-1, 1]^4$ concentrates volume along its diagonals, causing normalized samples to heavily cluster toward the 4D hypercube corners—corresponding to 16 clusters when projected onto SO(3) [8].

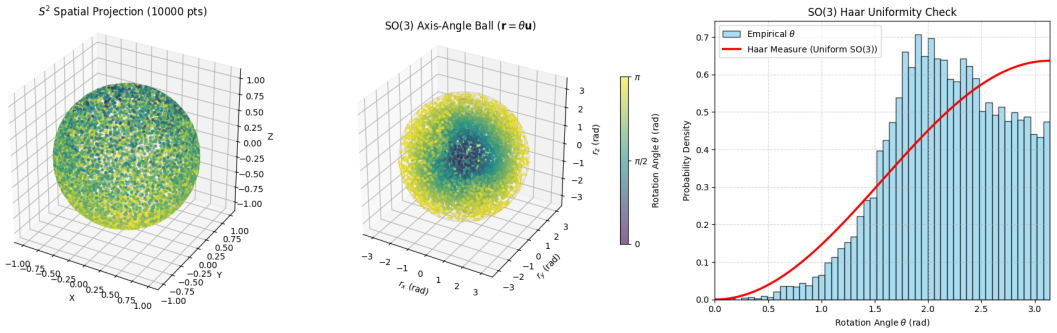
To ensure an unbiased uniform distribution over $SO(3)$, we employ the explicit uniform quaternion generation algorithm derived by Shoemake [12] and evaluated by Kuffner [6]. Given three independent uniform random variables $u_1, u_2, u_3 \sim \mathcal{U}[0, 1]$, a unit quaternion $\mathbf{q} = [w, x, y, z]^\top \in S^3$ is constructed directly as:

$$\mathbf{q} = \begin{bmatrix} \sqrt{1 - u_1}\sin(2\pi u_2) \\ \sqrt{1 - u_1}\cos(2\pi u_2) \\ \sqrt{u_1}\sin(2\pi u_3) \\ \sqrt{u_1}\cos(2\pi u_3) \end{bmatrix} \tag{1.72}$$

The resulting spatial and angular distributions are shown in Figs. 1.5a and 1.5b. The naive sampling method produces a highly non-uniform ditributions with clustering correlated to the corners of the 4D hypercube, reduced density of small angle samples, and major deviations when compared to the Haar measure curve.



(a) Uniform quaternion sampling.



(b) Naïve quaternion sampling.

Figure 1.5: Comparison of uniform and naïve quaternion sampling. The spatial projection, axis–angle distribution, and empirical rotation-angle distribution illustrate the effect of sampling method on the resulting distribution over SO(3).

This parametrization maps the 3D unit cube $[0, 1]^3$ directly onto the 3-sphere S^3 with uniform surface density, satisfying $\|\mathbf{q}\|_2 = 1$ identically without requiring normalization

[6, 8]. We account for the double-cover property of quaternions ($\mathbf{q} \sim -\mathbf{q}$) by mapping any sample located in the lower hemisphere ($w < 0$) to its antipodal equivalent $-\mathbf{q}$ without distorting the uniform density [8].

Kinodynamic Constraints

Constraints imposed on the OCP are divided into two categories: (initial or final) boundary conditions, and path constraints.

The collection of boundary condition equality constraints is straightforward:

- Orientation: $\mathbf{q}(t_0) = \mathbf{q}_0$, $\mathbf{q}(t_f) = \mathbf{q}_f$
- Angular velocity: $\boldsymbol{\omega}(t_0) = \mathbf{0}$, $\boldsymbol{\omega}(t_f) = \mathbf{0}$
- Angular acceleration: $\boldsymbol{\alpha}(t_0) = \mathbf{0}$, $\boldsymbol{\alpha}(t_f) = \mathbf{0}$
- Angular jerk: $\dot{\alpha}(t_0) = \mathbf{0}$, $\dot{\alpha}(t_f) = \mathbf{0}$ (for select cases)

To enforce the boundary conditions, the corresponding B-spline control vertices are solved deterministically.

Kinematic and dynamic path constraints consist of:

- Angular velocity limits: $\|\boldsymbol{\omega}(t)\|_\infty \leq \omega_{\max}$
- Torque limits: $\|\boldsymbol{\tau}(t)\|_\infty \leq \tau_{\max}$
- Orientation exclusion zone (EZ): $(R_{ib}{}^b\hat{\mathbf{s}})^\top {}^i\hat{\mathbf{e}} \leq \cos(\theta_{\text{exclusion}})$
- Quaternion unit norm: $0.998 \leq \|\mathbf{q}\|_2 \leq 1.002$,

where ${}^b\hat{\mathbf{s}}$ is the sensor direction vector, ${}^i\hat{\mathbf{e}}$ is the exclusion direction vector in the inertial coordinate frame, and $\theta_{\text{exclusion}}$ is the angular radius of the EZ. The quaternion unit norm equality constraint was relaxed to a narrow inequality constraint to improve the optimizer's ability to explore the search space where instead of a 3-D manifold, a thin 4-D shell is permitted.

1.3.2 7-DOF Robotic Manipulator Model

The robotic manipulator model describes the joint-space kinematics and dynamics of a redundant 7-DOF arm conisisting of revolute joints existing on a bounded joint space $\mathcal{Q} \subset \mathbb{R}^7$. Task space end-effector poses belong to SE(3). The optimization parameters are purely the B-spline control vertices for the joint-position trajectories, $\mathbf{q}(t)$, with all other kinematic (joint velocities, accelerations, jerks, end-effector poses, and Jacobians) and dynamic (joint torques) quantities derived directly from the joint trajectories and their derivatives.

Joint Trajectories and Their Derivatives

If the joint control vertex matrix is $\mathbf{P} \in \mathbb{R}^{7\times n_{\text{ctrl}}}$ and the B-spline basis vector evaluated over normalized time $s = \frac{t-t_0}{T} \in [0,1]$ is $\boldsymbol{\varphi}(s) \in \mathbb{R}^{n_{\text{ctrl}}\times 1}$, the joint position trajectory is given by:

$$\mathbf{q}(s) = \mathbf{P}\boldsymbol{\varphi}(s) \in \mathbb{R}^{7\times 1}, \tag{1.73}$$

or, evaluated across all discrete sampling times, in matrix form:

$$\mathbf{Q} = \mathbf{P}\boldsymbol{\Phi} \in \mathbb{R}^{7\times n_{\text{via}}}, \tag{1.74}$$

where $\mathbf{Q}$ is the matrix of joint positions at all via points, $\boldsymbol{\Phi} \in \mathbb{R}^{n_{\text{ctrl}}\times n_{\text{via}}}$ is the B-spline basis matrix evaluated across all via points, n_{ctrl} is the number of control vertices per joint, and n_{via} represents the number of evaluation points along the trajectory.

Because time-scaling to the total trajectory duration $T = t_{\text{catch}} - t_0$ introduces the differential factor $\frac{ds}{dt} = \frac{1}{T}$, higher-order time derivatives explicitly scale by powers of $\frac{1}{T}$. The joint velocity, acceleration, and jerk trajectories are evaluated directly via the derivative basis functions:

$$\dot{\mathbf{q}}(t) = \frac{1}{T}\mathbf{P}\dot{\boldsymbol{\varphi}}(s) \in \mathbb{R}^{7\times 1}, \tag{1.75}$$

$$\dot{\mathbf{Q}} = \frac{1}{T}\mathbf{P}\dot{\boldsymbol{\Phi}} \in \mathbb{R}^{7\times n_{\text{via}}}, \tag{1.76}$$

$$\ddot{\mathbf{q}}(t) = \frac{1}{T^2}\mathbf{P}\ddot{\boldsymbol{\varphi}}(s) \in \mathbb{R}^{7\times 1}, \tag{1.77}$$

$$\ddot{\mathbf{Q}} = \frac{1}{T^2}\mathbf{P}\ddot{\boldsymbol{\Phi}} \in \mathbb{R}^{7\times n_{\text{via}}}, \tag{1.78}$$

$$\dddot{\mathbf{q}}(t) = \frac{1}{T^3}\mathbf{P}\dddot{\boldsymbol{\varphi}}(s) \in \mathbb{R}^{7\times 1}, \tag{1.79}$$

$$\dddot{\mathbf{Q}} = \frac{1}{T^3}\mathbf{P}\dddot{\boldsymbol{\Phi}} \in \mathbb{R}^{7\times n_{\text{via}}}. \tag{1.80}$$

Equations of Motion (EOM)

The state vector of the robot is defined as $\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top$, where $\mathbf{q} \in \mathbb{R}^7$ is the vector of joint positions and $\dot{\mathbf{q}} \in \mathbb{R}^7$ is the vector of joint velocities.

The joint-space dynamics of the manipulator are governed by the EOM:

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q}), \tag{1.81}$$

where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{7\times 7}$ is the mass matrix, $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^7$ is the centripetal-Coriolis vector, $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^7$ is the gravity vector, and $\boldsymbol{\tau} \in \mathbb{R}^7$ represents the control torque *input* applied by the actuators.

Rearranging (1.81) for the joint acceleration yields:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}\left[\boldsymbol{\tau} - \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q})\right]. \tag{1.82}$$

The end-effector pose is obtained from the forward-kinematics:

$$\mathbf{T}_{0,8} = \left(\prod_{i=1}^{7} \mathbf{T}_{i-1,i}(q_i)\right)\mathbf{T}_{7,8} = \mathbf{T}_{0,7}(\mathbf{q})\mathbf{T}_{7,8} \tag{1.83}$$

$$\mathbf{T}_{i,ee}(\mathbf{q}) = \mathbf{T}_{i,0}\mathbf{T}_{0,7}(\mathbf{q})\mathbf{T}_{7,8}\mathbf{T}_{8,ee}, \tag{1.84}$$

where indices i, ee, and $n \in \{0, \ldots, 8\}$ refer to the inertial frame, the end-effector frame, and the individual joint frames as described by the MDH table, respectively. The final joint frame, denoted by $n = 8$, is fixed relative to joint frame $n = 7$ and does not depend on the joint configuration. The end-effector position and orientation are extracted as:

$$^{i}\mathbf{p}_e(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3, 4), \qquad \mathbf{R}_{i,ee}(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3, 1{:}3). \tag{1.85}$$

The corresponding end-effector twist, $^{i}\mathbf{v}_e(\mathbf{q}, \dot{\mathbf{q}})$ is related to the joint velocity by the geometric Jacobian $\mathbf{J}_e(\mathbf{q})$:

$$^{i}\mathbf{v}_e(\mathbf{q}, \dot{\mathbf{q}}) = {}^{i}\mathbf{J}_e(\mathbf{q})\dot{\mathbf{q}}, \tag{1.86}$$

and its acceleration, $^{i}\mathbf{a}_e(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$, is:

$$^{i}\mathbf{a}_e(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = {}^{i}\mathbf{J}_e(\mathbf{q})\ddot{\mathbf{q}} + {}^{i}\dot{\mathbf{J}}_e(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}. \tag{1.87}$$

For the interior of the OCP, where the trajectory is parameterized directly in joint coordinates using B-splines, the optimizer has the freedom to optimize the interior control vertices, $\mathbf{c}_{int}$. At each interior via point, the order of operations follows:

$$\boxed{\mathbf{c}_{int} \longrightarrow \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}} \longrightarrow \mathbf{M}(\mathbf{q}), \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}), \mathbf{g}(\mathbf{q}) \longrightarrow \boldsymbol{\tau}}.$$

At the trajectory boundaries, the B-spline parameterization deterministically enforces the kinematic boundary conditions via assignment of the B-spline control vertices. The trajectory is defined by n control vertices ($\mathbf{c}_0$ through $\mathbf{c}_{n-1}$). The initial conditions impose a fixed joint position, zero velocity, and zero acceleration. The final conditions impose zero velocity and zero acceleration, but leave the final joint position unconstrained so that it may be optimized to intercept the target.

As a result, five dependent control vertices ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_{n-3}, \mathbf{c}_{n-2}$) are deterministic. They are calculated as exact functions of the initial state constraints, the final velocity and acceleration constraints, and the optimized final position, which is directly represented by the final vertex ($\mathbf{c}_{n-1}$). This analytical reduction leaves the remaining $n - 5$ control vertices as the free decision variables provided to the optimizer:

Kinematics: $\boxed{\mathbf{q}_0, \dot{\mathbf{q}}_0, \ddot{\mathbf{q}}_0, \dot{\mathbf{q}}_f, \ddot{\mathbf{q}}_f, \mathbf{c}_{n-1} \longrightarrow \mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_{n-3}, \mathbf{c}_{n-2} \longrightarrow \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}}$

Dynamics: $\boxed{\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}} \longrightarrow \mathbf{M}(\mathbf{q}), \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}), \mathbf{g}(\mathbf{q}) \longrightarrow \boldsymbol{\tau}}$

Exterior Control Vertex Calculation

For the robotic manipulator model, each joint trajectory is represented by a cubic B-spline:

$$\mathbf{q}(t) = \mathbf{P}\boldsymbol{\varphi}(t), \qquad \mathbf{P} = \begin{bmatrix} \mathbf{c}_0 & \mathbf{c}_1 & \cdots & \mathbf{c}_{n_{\text{ctrl}}-1} \end{bmatrix} \in \mathbb{R}^{7 \times n_{\text{ctrl}}}, \tag{1.88}$$

where $\mathbf{q}(t) \in \mathbb{R}^7$ is the joint-position vector and each $\mathbf{c}_i \in \mathbb{R}^7$ contains one control vertex for each joint.

For a clamped cubic B-spline, the number of knot spans is

$$n_{\text{seg}} = n_{\text{ctrl}} - p, \qquad p = 3. \tag{1.89}$$

For a uniform knot vector, the knot-span duration is therefore

$$h = \frac{t_f}{n_{\text{seg}}}, \tag{1.90}$$

where t_f is the duration of the joint trajectory.

The initial boundary conditions are

$$\mathbf{q}(0) = \mathbf{q}_0, \qquad \dot{\mathbf{q}}(0) = \dot{\mathbf{q}}_0, \qquad \ddot{\mathbf{q}}(0) = \ddot{\mathbf{q}}_0. \tag{1.91}$$

For a clamped cubic B-spline, these conditions determine the first three control vertices:

$$\mathbf{c}_0 = \mathbf{q}_0, \tag{1.92}$$

$$\mathbf{c}_1 = \mathbf{c}_0 + \frac{h}{3}\dot{\mathbf{q}}_0, \tag{1.93}$$

$$\mathbf{c}_2 = 3\mathbf{c}_1 - 2\mathbf{c}_0 + \frac{h^2}{3}\ddot{\mathbf{q}}_0. \tag{1.94}$$

At the terminal boundary, the final velocity and acceleration are:

$$\dot{\mathbf{q}}(t_f) = \dot{\mathbf{q}}_f, \qquad \ddot{\mathbf{q}}(t_f) = \ddot{\mathbf{q}}_f. \tag{1.95}$$

The final control vertex is the final joint position when the ball is caught. The remaining final exterior vertices are determined from this vertex and the terminal derivative conditions:

$$\mathbf{c}_{n_{\text{ctrl}}-2} = \mathbf{c}_{n_{\text{ctrl}}-1} - \frac{h}{3}\dot{\mathbf{q}}_f, \tag{1.96}$$

$$\mathbf{c}_{n_{\text{ctrl}}-3} = \mathbf{c}_{n_{\text{ctrl}}-1} - h\dot{\mathbf{q}}_f + \frac{h^2}{3}\ddot{\mathbf{q}}_f. \tag{1.97}$$

Thus, the control vertices matrix is

$$\mathbf{P} = \begin{bmatrix} \mathbf{c}_0 & \mathbf{c}_1 & \mathbf{c}_2 & \mathbf{c}_3 & \cdots & \mathbf{c}_{n_{\text{ctrl}}-4} & \mathbf{c}_{n_{\text{ctrl}}-3} & \mathbf{c}_{n_{\text{ctrl}}-2} & \mathbf{c}_{n_{\text{ctrl}}-1} \end{bmatrix}. \tag{1.98}$$

For the ball-catching problem, the interior control vertices

$$\mathbf{c}_3, \ldots, \mathbf{c}_{n_{\text{ctrl}}-4}$$

and the terminal-position vertex

$$\mathbf{c}_{n_{\text{ctrl}}-1}$$

are the optimization variables. The remaining exterior vertices are reconstructed analytically from the boundary conditions.

Because the terminal joint position is not set directly in the ball-catching problem, $\mathbf{c}_{n_{\text{ctrl}}-1}$ is determined indirectly by the terminal end-effector and approach-axis constraints.

For a fixed-endpoint problem, the terminal joint position is prescribed:

$$\mathbf{c}_{n_{\text{ctrl}}-1} = \mathbf{q}_f. \tag{1.99}$$

In that case, all six exterior control vertices would be determined by the boundary conditions, as shown for the spacecraft model, where only the interior control vertices would remain as optimization variables.

State and Control Vertex Bounds

The state space for the robotic manipulator is simply bounded by the joint position limits $[q_{i,\min}, q_{i,\max}], i \in \{1, \ldots, 7\}$. However, unlike the spacecraft model, the robotic manipulator does not have an easily defined space for the control vertex bounds. This is because, while the B-spline convex hull property applies, there is no guaranteed lower bound on the ratio of trajectory position to control vertex value; the control vertex values may be substantially greater than the individual joint limits while still producing feasible trajectories. As the aim of this work is to perform *global* trajectory optimization, it is critical to derive a set of control vertex bounds that do not arbitrarily exclude feasible local minima. Therefore, it is necessary to derive the bounds using B-spline principles and model specific details.

Control Vertex Bounds. Let the joint position trajectory, $q(t)$, be a clamped, uniform, cubic ($p = 3$) B-spline with $N = 11$ control vertices. There are $N_T = N + p + 1 = 15$ knots and $N - p = 8$ internal knot spans. To ensure the NLP's search space covers all dynamically valid trajectories up to the maximum solve time, $t_{f,\max} = 1.8\,\text{s}$, the bounds must be calculated using the maximum possible knot span:

$$\Delta t_{\max} = \frac{t_{f,\max}}{N - p} = \frac{1.8}{8} = 0.225\,\text{s}. \tag{1.100}$$

The joint velocity, $\dot{q}(t)$, is a quadratic B-spline defined by velocity control vertices, v_i. From the convex hull property of B-splines, $|\dot{q}(t)|$ is bounded by the maximum velocity vertex $|v_i|$. Therefore, the following relation must be satisfied:

$$|v_i| \leq \dot{q}_{\max}. \tag{1.101}$$

Edit this to not have set number of CVs and time

Velocity Control Vertex Formula: For a B-spline with degree p and knot vector $T = \{t_1, t_2, \ldots, t_{N+p+1}\}$, the velocity control vertices are given by:

$$v_i = \frac{p}{t_{i+p+1} - t_{i+1}}(c_{i+1} - c_i), \tag{1.102}$$

where c_i are the joint position control vertices of the B-spline.

Endpoint Vertices (c_1, c_{11}): For clamped B-splines, $c_1 = q_0$ and $c_{11} = q_f$. The bounds on endpoint vertices are simply the position bounds:

$$B_{\max,i} = q_{\max}, \quad B_{\min,i} = q_{\min}, \quad i \in \{1, 11\}. \tag{1.103}$$

Interior Vertices ($c_3, \ldots, c_9$ and conservatively c_2, c_{10}): Evaluating the velocity control vertex formula for degree $p = 3$:

$$v_i = \frac{3}{t_{i+4} - t_{i+1}}(c_{i+1} - c_i) = \frac{3}{4\Delta t_{\max} - \Delta t_{\max}}(c_{i+1} - c_i) = \frac{1}{\Delta t_{\max}}(c_{i+1} - c_i). \tag{1.104}$$

Constraining via $|v_i| \leq \dot{q}_{\max}$ yields:

$$|c_{i+1} - c_i| \leq \dot{q}_{\max} \Delta t_{\max}, \tag{1.105}$$

which provides the control vertex bounds:

$$B_{\max,i} = q_{\max} + \dot{q}_{\max}\Delta t_{\max}, \qquad B_{\min,i} = q_{\min} - \dot{q}_{\max}\Delta t_{\max}, \tag{1.106}$$

for $i \in \{3, \ldots, 9\}$ While $i \in \{2, 10\}$ can be given tighter bounds due to knot multiplicity at the clamped endpoints, the optimizer does not optimize the control vertices corresponding to these indices. As a simplified and conservative approach, the same looser bounds may be applied to this control vertices as well.

Additionally, the search space could be further constrained by enforcing independent lower and upper bounds for each joint position, rather than the joint-specific symmetric bounds used here. ■

Kinodynamic Constraints

As in the spacecraft model, constraints imposed on the robot model consist of boundary and path constraint.

The collection of boundary condition constraints is:

- Initial joint position: $\mathbf{q}(t_0) = \mathbf{q}_{\text{initial}}$
- Joint velocity: $\dot{\mathbf{q}}(t_0) = \mathbf{0}$, $\dot{\mathbf{q}}(t_f) = \mathbf{0}$
- Joint acceleration: $\ddot{\mathbf{q}}(t_0) = \mathbf{0}$, $\ddot{\mathbf{q}}(t_f) = \mathbf{0}$
- Terminal end-effector position: ${}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q}(t_f)) - {}^{i}\mathbf{p}_b(t_f) = \mathbf{0}$
- Terminal end-effector orientation: $\arcsin({}^{i}a_{\text{ee},z}) - \arcsin({}^{i}a_{b,z}) = 0$ and $\operatorname{atan2}({}^{i}a_{\text{ee},y}, {}^{i}a_{\text{ee},x}) - \operatorname{atan2}({}^{i}a_{b,y}, {}^{i}a_{b,x}) = 0$
- Catch time: $0.06 \leq t_f \leq 1.8$, where the spline duration is $T = t_f - t_0 > 0$,

where ${}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q}(t_f))$ and ${}^{i}\mathbf{p}_b(t_f)$ are the positions of the end-effector and the ball at the catch time, respectively, ${}^{i}\mathbf{a}_{\text{ee}}(\mathbf{q}(t_f))$ is the unit vector of the end-effector approach axis—the axis pointing out of the palm of the robotic hand—at the terminal joint configuration, and ${}^{i}\mathbf{a}_b(t_f)$ is the unit vector of the ball approach axis—equivalent to the direction of the ball's velocity—at the catch time.

${}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q})$ and $\mathbf{R}_{\text{i,ee}}(\mathbf{q})$ are components retrieved from the homogenous transformation matrix $\mathbf{T}_{\text{i,ee}}(\mathbf{q})$ obtained via forward kinematics:

$$\mathbf{T}_{\text{i,ee}}(\mathbf{q}) = \begin{bmatrix} \mathbf{R}_{\text{i,ee}}(\mathbf{q}) & {}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q}) \\ \mathbf{0}^{\top} & 1 \end{bmatrix}. \tag{1.107}$$

$\mathbf{T}_{\text{i,ee}}\mathbf{q}$ is obtained via the chain of homogeneous transformation matrices from the base frame to the end-effector frame: $\mathbf{T}_{\text{i,ee}}(\mathbf{q}) = \mathbf{T}_{i,B}\mathbf{T}_{B,F}(\mathbf{q})\mathbf{T}_{\text{F,ee}}$, where i denotes the inertial frame, B denotes the robot base frame, F denotes the joint dependent MDH frame, and ee denotes the end-effector frame.

Unlike the directly specified terminal orientations in the spacecraft model, the robot's terminal joint position $\mathbf{q}(t_f)$ is not fixed. Instead, it is implicitly determined by the end-effector pose constraints required to intercept the target ball. The ball position at the

catch time is defined analytically using the ballistic equation ${}^{i}\mathbf{p}_b(t_f) = {}^{i}\mathbf{p}_{b,0} + t_f {}^{i}\mathbf{v}_{b,0} + \frac{1}{2}t_f^2 {}^{i}\mathbf{g}$, yielding three scalar equality constraints for position.

To ensure a successful catch, the end-effector approach axis ${}^{i}\mathbf{a}_{\text{ee}}(\mathbf{q}(t_f))$—extracted from the third column of the forward-kinematics rotation matrix $\mathbf{R}_{\text{i,ee}}(\mathbf{q}(t_f))$—must align (specifically, be anti-parallel) with the ball approach axis::

$$ {}^{i}\mathbf{a}_b(t_f) = -\frac{{}^{i}\mathbf{v}_{b,0} + t_f {}^{i}\mathbf{g}}{\|{}^{i}\mathbf{v}_{b,0} + t_f {}^{i}\mathbf{g}\|} \tag{1.108} $$

This alignment is enforced via the two orientation constraints defined in the list above, utilizing the x, y, and z components of the respective approach vectors. Together, the terminal end-effector pose contributes a total of five scalar equality constraints (3 position, 2 orientation) to the boundary conditions. For the 7-DOF robot, the five terminal end-effector pose constraints leave two additional degrees of freedom in the joint space. The redundancy results in infinite possible joint configurations satisfying the terminal end-effector pose constraints and allows for the global search of optimal trajectories.

Kinematic and dynamic path constraints consist of:

- Joint position limits: $\mathbf{q}_{\min} \leq \mathbf{q}(t) \leq \mathbf{q}_{\max}$
- Joint velocity limits: $\dot{\mathbf{q}}_{\min} \leq \dot{\mathbf{q}}(t) \leq \dot{\mathbf{q}}_{\max}$
- Torque limits: $\boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}(t) \leq \boldsymbol{\tau}_{\max}$.

There is a conspicuous absence of obstacle avoidance constraints in the current formulation, particularly concerning self-collisions. However, previous work by [7] showed self-collisions never occurred in practice; given this model is a replication of that work, it is reasonable to assume self-collisions will similarly not occur here.

1.4 Optimal Control Problem (OCP) & Nonlinear Programming (NLP) Formulation

The objective of the algorithms posed in this work is to generate optimal trajectories for the spacecraft and robotic manipulator models. In order to obtain an optimal result, the objective must first be formalized into an OCP followed by a transcription to an NLP. The transcription involves discretizing the continuous-time problem into a finite set of state, control, and time nodes through the use of B-splines. A myriad of transcription methods exist including direct collocation, indirect and direct shooting, and pseudospectral methods. For a robust review of the commonly used transcription methods, refer to [2, 10]. The method employed in this work is direct collocation via B-spline parameterization which provides a very sparse structure for the resulting nonlinear program, facilitating efficient optimization. Following discretization, a solver must be employed, over which several types exist; here we use IPOPT as the underlying *interior-point* NLP solver.

1.5 Optimal Control Problem

To evaluate the B-spline trajectory optimization framework across different dynamical systems, two physical models are studied as benchmarks: a 3-DOF rigid-body spacecraft attitude slew and a 7-DOF robotic manipulator ball-catching maneuver.

1.5.1 Problem 1: Spacecraft Attitude Slew Maneuver

The first problem considers a two-point boundary value problem (BVP) for a spacecraft attitude slew maneuver over a fixed time horizon. The specifications on the state vector, constraints, and EOM are detailed in § 1.3.1.

Continuous-Time OCP Statement

The continuous-time spacecraft OCP minimizes control effort:

$$\min_{\mathbf{q}(t)} \quad J_{\text{spacecraft}} = \frac{1}{w_{\text{scale}}} \int_{t_0}^{t_f} \|\boldsymbol{\tau}(t)\|_2^2 \, \mathrm{d}t, \tag{1.109}$$

subject to:

$$\text{Inverse Kinodynamics:} \quad \boldsymbol{\omega}(t) = 2\mathbf{C}(\mathbf{q}(t))\dot{\mathbf{q}}(t), \tag{1.110}$$

$$\boldsymbol{\tau}(t) = \mathbf{I}\dot{\boldsymbol{\omega}}(t) + \boldsymbol{\omega}(t) \times (\mathbf{I}\boldsymbol{\omega}(t)), \tag{1.111}$$

$$\text{Quaternion Unit Norm:} \quad \|\mathbf{q}(t)\|_2 - 1 = 0, \tag{1.112}$$

$$\text{Actuator Torque Limits:} \quad \boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}(t) \leq \boldsymbol{\tau}_{\max}, \tag{1.113}$$

$$\text{Angular Velocity Limits:} \quad \boldsymbol{\omega}_{\min} \leq \boldsymbol{\omega}(t) \leq \boldsymbol{\omega}_{\max}, \tag{1.114}$$

$$\text{Initial Rest Boundary:} \quad \mathbf{q}(t_0) = \mathbf{q}_0, \quad \boldsymbol{\omega}(t_0) = \mathbf{0}, \quad \dot{\boldsymbol{\omega}}(t_0) = \mathbf{0}, \quad \ddot{\boldsymbol{\omega}}(t_0) = \mathbf{0}^{\dagger}, \tag{1.115}$$

$$\text{Final Rest Boundary:} \quad \mathbf{q}(t_f) = \mathbf{q}_f, \quad \boldsymbol{\omega}(t_f) = \mathbf{0}, \quad \dot{\boldsymbol{\omega}}(t_f) = \mathbf{0}, \quad \ddot{\boldsymbol{\omega}}(t_f) = \mathbf{0}^{\dagger}. \tag{1.116}$$

† enforced for select test cases.

1.5.2 Problem 2: 7-DOF Manipulator Ball-Catching Maneuver

The second problem considers an 7-DOF robotic manipulator catching a ball over a free time horizon $t \in [0.06, t_f]$s, with the terminal catch time t_f and end-effector pose are free decision variables. The specifications on the state vector, constraints, and EOM are detailed in § 1.3.2.

Continuous-Time OCP Statement

The continuous-time optimal control problem for the 7-DOF manipulator ball catch minimizes joint control effort and jerk over the free catch time $t_f \in [0.06, 1.8]$ s:

$$\min_{\mathbf{q}(t), t_f} \quad J_{\text{robot}} = \int_{t_0}^{t_f} \left(\sum_{i=1}^{7} (\tau_i(t)\dot{q}_i(t))^2 + \|\mathbf{J}\,\dddot{\mathbf{q}}(t)\|_2^2 \right) \mathrm{d}t, \tag{1.117}$$

where $\mathbf{J}$ is the diagonal matrix of joint motor inertias,
subject to:

$$\begin{aligned}
\text{Inverse Kinodynamics:} \quad & \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}(t), && (1.118)\\
& \mathbf{p}_{ee}(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3,4), && (1.119)\\
& \mathbf{R}_{i,ee}(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3,1{:}3), && (1.120)\\
\text{Joint Position Bounds:} \quad & \mathbf{q}_{\min} \le \mathbf{q}(t) \le \mathbf{q}_{\max}, && (1.121)\\
\text{Joint Velocity Bounds:} \quad & \dot{\mathbf{q}}_{\min} \le \dot{\mathbf{q}}(t) \le \dot{\mathbf{q}}_{\max}, && (1.122)\\
\text{Actuator Torque Bounds:} \quad & \boldsymbol{\tau}_{\min} \le \boldsymbol{\tau}(t) \le \boldsymbol{\tau}_{\max}, && (1.123)\\
\text{Initial Rest Boundary:} \quad & \mathbf{q}(t_0) = \mathbf{q}_0, \quad \dot{\mathbf{q}}(t_0) = \mathbf{0}, \quad \ddot{\mathbf{q}}(t_0) = \mathbf{0}, && (1.124)\\
\text{Final Rest Boundary:} \quad & \dot{\mathbf{q}}(t_f) = \mathbf{0}, \quad \ddot{\mathbf{q}}(t_f) = \mathbf{0}, && (1.125)\\
\text{Final Catch Position:} \quad & \mathbf{p}_{\text{ee}}(\mathbf{q}(t_f)) = \mathbf{p}_{ball}(t_f), && (1.126)\\
\text{Final Approach Alignment:} \quad & \text{approach-axis}(\mathbf{q}(t_f)) - \text{ball-axis}(t_f) = 0, && (1.127)\\
\text{Catch Time Horizon Bounds:} \quad & 0.06 \le t_f \le 1.8\text{s}, && (1.128)
\end{aligned}$$

with (1.127) defined in § 1.3.2 and all points and vectors expressed in the inertial frame unless otherwise noted.

1.5.3 NLP Decision Variable Structure

When transcribed using B-splines, both continuous problems map the boundary state constraints that are known a priori directly onto the corresponding exterior control vertices ($\mathbf{P}_{\text{ext}}$)—which are evaluated
analytically. For the spacecraft, only the interior control vertices $\mathbf{P}_{\text{int}}$ remain as free decision variables. For the robotic manipulator, the interior control vertices, the final control vertex $\mathbf{P}_f$, and the final time t_f are free decision variables in the optimization vector $\mathbf{w}$:

- **Spacecraft Slew Decision Vector:** Parameterized by 4 quaternion elements (n_{ctrl} control vertices per element). With boundary conditions embedded, the number of free variables is reduced by the number of boundary constraints, $n_{\text{free}} = n_{\text{ctrl}} - n_{\text{bc}}$:

$$\mathbf{w}_{\text{spacecraft}} = \mathbf{P}_{\text{int,sc}} \in \mathbb{R}^{4n_{\text{free}}}. \tag{1.129}$$

  n_{bc} may equal either six or eight, depending on whether angular jerk is included in the boundary conditions.

- **7-DOF Robotic Arm Decision Vector:** Parameterized by 7 B-spline joint curves (n_{ctrl} control vertices per curve) with $n_{\text{free}} = n_{\text{ctrl}} - n_{\text{bc}}$ free interior control vertices per joint:

$$\mathbf{w}_{\text{robot}} = \begin{bmatrix} \mathbf{P}_{\text{int}} & \mathbf{P}_f & t_f \end{bmatrix} \in \mathbb{R}^{7n_{\text{free}}+1}. \tag{1.130}$$

  n_{bc} equals five; the boundary conditions on catch position, approach angle, and time do not reduce the number of decision variables because the final control vertex is not constrained to a single configuration.

1.5.4 NLP Optimization Framework

The transcribed finite-dimensional NLPs are solved using the primal-dual interior-point solver IPOPT developed by Wächter and Biegler at Carnegie Mellon University for the solving of nonlinear, nonconvex optimization problems [16]. IPOPT is released as open-source software [16]. More can be found in the original publication by Wächter and Biegler [16]. The solver minimizes the objective function subject to path and boundary constraints by solving a sequence of barrier subproblems using a filter line-search method, with local optima satisfying the Karush-Kuhn-Tucker (KKT) conditions. Exact first- and second-order derivatives—specifically the constraint Jacobian $\nabla \mathbf{g}(\mathbf{w})$ and the Hessian of the Lagrangian $\nabla^2 \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda})$—are evaluated analytically via automatic differentiation using CasADi [1]. Because B-splines possess local support, each decision variable influences only a small portion of the trajectory, i.e., a limited subset of collocation nodes, resulting in highly sparse, banded matrices that IPOPT exploits for computationally efficient linear solves.

1.6 Algorithm Frameworks

Present the four comparative trajectory-generation approaches with a consistent structure:

- ☐ **MC-GBO:** Monte Carlo Gradient-Based Optimization.
- ☐ **RRT*-GBO:** Rapidly-exploring Random Tree Star combined with Gradient-Based Optimization.
- ☐ **P-HO:** Probabilistic Homotopy Optimization.
- ☐ **IPSO:** Inverse Dynamics Particle Swarm Optimization.
- ☐ For each approach, describe the initialization, sampling or warm-start mechanism, optimization phase, constraint treatment, and resulting trajectory representation.
- ☐ Include the SO(3) sampling mechanisms, distinguishing naive sampling from uniform sampling on SO(3).
- ☐ Include the sampling bounds used for the Arm7 manipulator.

1.7 Algorithm Frameworks

The core of this work is the implementation, evaluation, and comparison of global trajectory optimization approaches for realistic physical models and scenarios in aerospace and robotic applications. The spacecraft and robotic manipulator models and tasks prove highly nonlinear and are constrained by complex kinodynamic and environmental limitations. After a review of the SOTA of *kinodynamically-consistent* trajectory optimization methods, four algorithms are selected and introduced:

- Monte Carlo Gradient-Based Optimization (MC-GBO),

- Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO) [15],
- Probabilistic Homotopy Optimization (P-HO) [9],
- Inverse Dynamics Particle Swarm Optimization (IPSO) [13, 14].

This selection encompasses a diverse set of strategies—spanning naive multistart, sampling, homotopy/continuation, and metaheuristic optimization approaches, as well as a combination of these paradigms. Each algorithm is evaluated in terms of its ability to generate optimal trajectories while satisfying kinodynamic and environmental constraints. Primary evaluation criteria include solution quality, convergence speed, and robustness under varying test case scenarios. Each algorithm is further detailed in the following subsections.

1.7.1 Monte Carlo Gradient-Based Optimization (MC-GBO)

Write MC-GBO details

Algorithm 1.1: `MC_GBO`

Data: A base random seed s_0 used to generate independent initial trajectory guesses.
Result: Parallel optimization of trajectories to find the minimum cost path until some iteration limit, N_{MC}, desired number of solutions, $N_{sol,des}$, or maximum time T_{max}.

$c_{best} \leftarrow \infty$;
$\mathbf{x}_{best} \leftarrow \emptyset$;
$N_{sol} \leftarrow 0$;
for $i \leftarrow 0$ **to** $N_{MC} - 1$ **do (optionally in parallel)**
  if $T \geq T_{max}$ **then**
    exit loop;
  $s_i \leftarrow s_0 + i$;
  $\mathbf{x}_{init} \leftarrow$ `random_initial_guess`(s_i);
  $(converged, \mathbf{x}_{opt}, c) \leftarrow$ `gbo_solve`$(\mathbf{x}_{init})$;
  if *converged* **then**
    N_{sol}++;
    if $c < c_{best}$ **then**
      $c_{best} \leftarrow c$;
      $\mathbf{x}_{best} \leftarrow \mathbf{x}_{opt}$;
    if $N_{sol} = N_{sol,des}$ **then**
      exit loop;
return $(\mathbf{x}_{best}, c_{best})$;

1.7.2 Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO)

Write RRT*-GBO details

Algorithm 1.2: `RRT*_GBO`

Data: A tree $\mathcal{T}$ which stores states as nodes, x, and trajectories as edges, e, containing a start state.
Result: Incremental building of the tree until some iteration limit, N, desired number of solutions, $N_{sol,des}$, or maximum time T_{max}.

```
for i ← 0 to N do
    if T ≥ T_max then
        exit loop;
    connect_to_goal(x_last, x_goal);
    x_new ← random_generator();
    if x_new ∉ R_obstacle then
        x_near ← neighbor_query(x_new, x_near, r_prune, tree);
        if e_new converged then
            add_to_tree(e_new, x_new);
            rewire_query(x_near, x_new);
```

Algorithm 1.3: `connect_to_goal`(x_{last}, x_{goal})

```
e_goal ← gbo_extend(x_last → x_goal);
if e_goal converged then
    if metric(e_goal) ≤ r_connect then
        add_to_tree(e_goal, x_goal);
        N_sol++;
        if N_sol = N_sol,des then
            exit loop;
```

Algorithm 1.4: `neighborhood_query`$(x_{new}, r_{prune}, tree)$

```
for j ← 0 to length_of(tree) do
    e_j ← gbo_extend(tree.x[j] → x_new);
    if metric(e_j) ≥ r_prune then
        if metric(e_j) ≤ r_ball then
            x_near[j] = x[j] ;                    // store x[j] ∈ r_ball
            if metric(e_j) < metric(e_new) then
                e_new = e_j;
add_to_tree(e_new);
```

Algorithm 1.5: `rewire_query`$(x_{new}, \boldsymbol{x}_{near})$

```
for j ← 0 to length_of(x_near) do
    e_j ← gbo_extend(x_new → x_near[j]);
    if metric(e_j) < metric(e_near,incoming[j]) then
        remove_from_tree(e_near,incoming[j]);
        add_to_tree(e_j);
```

informed pseudocode in progress

Algorithm 1.6: `informed_acceptance_filter` (Optional Variant)

Data: Candidate sample x_{rand}, nearest node x_{near}, goal x_f, best raw cost $c_{\text{best}}^{\text{raw}}$.
Result: **true** if sample is accepted, **false** if rejected.

if ***not*** `use_informed_sampling` ***or*** $c_{\text{best}}^{\text{raw}} = \infty$ **then**
  return true; `// Standard RRT*-GBO (uninformed) pass`
$\theta_{0f} \leftarrow \textbf{angle}(\mathcal{T}.x[\textbf{root}], x_f)$; **if** $\theta_{0f} \leq 0$ **then**
  return true;
$u \sim \mathcal{U}(0,1)$; **if** $u < \varepsilon_{\text{greedy}}$ **then**
  return true; `// Greedy exploration override`
$g_{\text{estimate}} \leftarrow \textbf{cost}(x_{near})$; $h_{\text{estimate}} \leftarrow \left(\frac{\textbf{angle}(x_{near},x_{rand})+\textbf{angle}(x_{rand},x_f)}{\theta_{0f}}\right)^2$;
$f_{\text{estimate}} \leftarrow g_{\text{estimate}} + h_{\text{estimate}}$;
$c_{\text{threshold}} \leftarrow c_{\text{best}}^{\text{raw}} s_{\text{margin}}$;
if $f_{\text{estimate}} < c_{\text{threshold}}$ **then**
  return true; `// Cost-based acceptance`
else
  return false; `// Informed rejection`

1.7.3 Probabilistic Homotopy Optimization (P-HO)

Write PHO details

Algorithm 1.7: `P_HO`

Data: Initial guess $\mathbf{x}_0$, iteration limit N, solve/sample threshold $\rho_{\mathcal{A}}$, and an optimization tree $\mathcal{T}$.
Result: A set of optimal trajectories corresponding to the goal problem parameterization $\mathbf{1}_d$.

$\mathcal{T} \leftarrow \{(\texttt{solve}(\mathcal{P}_{\Gamma(\mathbf{0}_d)}, \mathbf{x}_0), \mathbf{0}_d)\}$;
$\mathcal{S}_\lambda \leftarrow \{\mathbf{1}_d, \mathbf{0}_d\}$;
$\mathcal{S}_{\mathcal{A}} \leftarrow \emptyset$;
for $k \leftarrow 1$ **to** N **do**
  if $\frac{|\mathcal{S}_{\mathcal{A}}|}{|\mathcal{T}|\times|\mathcal{S}_\lambda|} \leq \rho_{\mathcal{A}}$ **then**
    $(\mathbf{N}_{i,j}, \boldsymbol{\lambda}_l) \leftarrow \texttt{sampleAttempt}(\mathcal{T}, \mathcal{S}_\lambda, \mathcal{S}_{\mathcal{A}})$;
    $\mathcal{S}_{\mathcal{A}} \leftarrow \mathcal{S}_{\mathcal{A}} \cup (\mathbf{N}_{i,j}, \boldsymbol{\lambda}_l)$;
    $\mathbf{x}^*_{new} \leftarrow \texttt{solve}(\mathcal{P}_{\Gamma(\boldsymbol{\lambda}_l)}, \mathbf{x}^*_i)$;
    if $\mathbf{x}^*_{new} \in \mathcal{C}$ ***and*** $\mathbf{x}^*_{new} \neq \mathbf{x}^*_j \; \forall \mathbf{N}_{j,\cdot} \in \mathcal{T}$ **then**
      $\mathcal{T} \leftarrow \mathcal{T} \cup (\mathbf{x}^*_{new}, \boldsymbol{\lambda}_l)$;
  else
    $\mathcal{S}_\lambda \leftarrow \mathcal{S}_\lambda \cup \texttt{sample}(\mathcal{H}_d)$;
return $\{\mathbf{x}^*_i : \mathbf{N}_{i,j} \in \mathcal{T}, \boldsymbol{\lambda}_j = \mathbf{1}_d\}$;

Algorithm 1.8: `sampleAttempt`

if *with probability* P_g **then**
  $\mathbf{N} \leftarrow \texttt{sample}(\{\mathbf{N}_{i,j} \in \mathcal{T} : (\mathbf{N}_{i,j}, \mathbf{1}_d) \notin \mathcal{S}_{\mathcal{A}}\})$;
  return $(\mathbf{N}, \mathbf{1}_d)$;
else
  return $\texttt{sample}(\{(\mathbf{N} \in \mathcal{T}, \boldsymbol{\lambda} \in \mathcal{S}_\lambda) : (\mathbf{N}, \boldsymbol{\lambda}) \notin \mathcal{S}_{\mathcal{A}}\})$;

1.7.4 Inverse Dynamics Particle Swarm Optimization (IPSO)

Write IPSO details

Algorithm 1.9: `IPSO_solve`

Data: Initialization of constants, swarm and tolerances.
Result: Optimized global best trajectory P^{global} and its final fitness f^{global}.

`initialize_swarm`(P_0, s_r);
$\delta \leftarrow \delta_0$;
for $o \leftarrow 0$ **to** $N_{outer} - 1$ **do**
  for $i \leftarrow 0$ **to** $N_{inner} - 1$ **do**
    for $j \leftarrow 1$ **to** $N_{particles}$ **do**
      $f_j \leftarrow$ `evaluate_fitness`(P_j, δ);
      if $f_j < f_j^{best}$ **then**
        $P_j^{best} \leftarrow P_j$;
        $f_j^{best} \leftarrow f_j$;
    $\left(P^{local}, P^{global}, f^{global}\right) \leftarrow$ `update_neighborhood_bests`(P^{best}, f^{best});
    $(w, c_{local}, c_{global}) \leftarrow$ `compute_scheduled_weights`(o, i);
    for $j \leftarrow 1$ **to** $N_{particles}$ **do**
      $r_p, r_l, r_g \sim \mathcal{U}(0, 1)$;
      $V_j \leftarrow$ `update_velocity`();
      $P_j, V_j \leftarrow$ `clamp`();
  $\delta_{new} \leftarrow$ `update_tolerance`(P^{global}, δ);
  if $\delta_{new} < \delta$ **then**
    `reset_swarm_memories`() ; `// Reset bests`
    $\delta \leftarrow \delta_{new}$;
  if *check_convergence*$(P^{global}, H, K, \varepsilon_{conv})$ **then**
    exit loop;
return (P^{global}, f^{global});

Algorithm 1.10: `IPSO_GBO`

Data: Warmstart trajectories from the IPSO runs for GBO refinement.
Result: Best optimized trajectory $\mathbf{x}_{best}$ and its corresponding minimum cost c_{best}.

$\mathcal{N} \leftarrow \emptyset$;
for $r \leftarrow 0$ **to** $N_{runs} - 1$ **do in parallel**
  $s_r \leftarrow s_0 + r$;
  $(P_{opt}, c_f) \leftarrow$ `IPSO_solve`(P_0, s_r);
  $\mathcal{N} \leftarrow \mathcal{N} \cup \{(P_{opt}, c_f)\}$;
$\mathcal{N}_{feasible} \leftarrow$ `sort_and_filter_feasible`$(\mathcal{N})$;
for $k \leftarrow 1$ **to** N_{ws} **do**
  $\mathbf{x}_{warm} \leftarrow \mathcal{N}_{feasible}[k].P_{opt}$;
  $(converged, \mathbf{x}_{opt}, c) \leftarrow$ `gbo_solve`$(\mathbf{x}_{warm})$;
  if *converged* ***and*** $c < \mathcal{N}_{feasible}[k].c_f$ **then**
    $\mathcal{N}_{feasible}[k] \leftarrow (\mathbf{x}_{opt}, c)$; ; `// Overwrite with refined GBO solution`
$(\mathbf{x}_{best}, c_{best}) \leftarrow$ `select_minimum_cost`$(\mathcal{N}_{feasible})$;
return $(\mathbf{x}_{best}, c_{best})$;

1.8 Adaptive Constraint Refinement

- ☐ Detail the iterative refinement pipeline for B-spline trajectories.
- ☐ Describe how kinodynamic and boundary-constraint violations are detected along the continuous B-spline trajectories.
- ☐ Describe how time-grid nodes are locally densified around detected violations.
- ☐ Explain how local refinement enforces feasibility without global over-discretization.

1.9 Analytical Verification

- ☐ **Analytical proof / baseline verification:** Present the unconstrained SO(3) maneuver proof.
- ☐ Use this proof as a mathematical validation that the baseline formulation and solver setup yield exact optimal solutions before evaluating complex constrained scenarios in Chapter 5.

Appendix A

Notation

A.1 Special operators and symbols

Identity matrix

$$\boldsymbol{I}_{n\times n} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \tag{A.1}$$

Skew-symmetric matrix

$$[\mathbf{v}]_\times = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix} \tag{A.2}$$

Inner product

$$\langle \boldsymbol{u}, \boldsymbol{v} \rangle = \boldsymbol{u}^\mathsf{T} \boldsymbol{v} \tag{A.3}$$

Cross product

$$\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_\times \mathbf{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix} \tag{A.4}$$

Mathematical symbols

j imaginary unit

$\boldsymbol{I}$ identity matrix

$\mathbf{0}$ zero matrix

$\boldsymbol{P}$ permutation matrix

$\boldsymbol{Q}$, $\boldsymbol{U}$ unitary (orthogonal) matrix

$\boldsymbol{T}$ transformation matrix

$\boldsymbol{R}(\varphi)$ rotation matrix

$\mathbb{R}^n$ n-dimensional real space

SO(3) special orthogonal group of three-dimensional rotations

$\mathfrak{so}(3)$ Lie algebra of SO(3)

SE(3) special Euclidean group of three-dimensional rigid-body motions

$\mathfrak{se}(3)$ Lie algebra of SE(3)

${}^A\boldsymbol{R}_B$ rotation matrix that maps coordinates from frame B to frame A

${}^A\boldsymbol{p}_B$ position of the origin of frame B expressed in frame A

${}^A\boldsymbol{T}_B$ homogeneous transformation from frame B to frame A

$$ {}^A\boldsymbol{T}_B = \begin{bmatrix} {}^A\boldsymbol{R}_B & {}^A\boldsymbol{p}_B \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix}. \tag{A.5} $$

$\boldsymbol{q} = [q_w, q_x, q_y, q_z]^\mathsf{T}$ unit quaternion, with $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$

$\overline{\boldsymbol{q}}$ quaternion conjugate

$\widehat{\boldsymbol{v}}$ skew-symmetric matrix associated with vector $\boldsymbol{v}$

Craig modified Denavit–Hartenberg parameters

a_{i-1} link length

α_{i-1} link twist

d_i link offset

θ_i joint angle

In the Craig modified Denavit–Hartenberg convention, the transform from frame i to frame $i-1$ is

$$ {}^{i-1}\boldsymbol{T}_i = \mathrm{Rot}_x(\alpha_{i-1})\,\mathrm{Trans}_x(a_{i-1})\,\mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i). \tag{A.6} $$

A.2 Abbreviations and acronyms

3-D three dimensional space

4-D four dimensional space

SE(3) special Euclidean group in three dimensions

SO(3) special orthogonal group in three dimensions

BVP boundary value problem

DCM direction cosine matrix

DH Denavit-Hartenberg

DOF degree of freedom

EOM equations of motion

EZ exclusion zone

GBO Gradient-Based Optimization

IPOPT Interior Point OPTimizer

IPSO Inverse-dynamics Particle Swarm Optimization

KKT Karush-Kuhn-Tucker

MC-GBO Monte Carlo with Gradient-Based Optimization

MDH Modified Denavit-Hartenberg

MRBE Mechatronics, Robotics, and Biomechanical Engineering

MRPs Modified Rodrigues Parameters

NLP nonlinear program

NLP nonlinear programming

OCP optimal control problem

P-HO Probabilistic Homotopy Optimization

PSO Particle Swarm Optimization

RRT Rapidly-exploring Random Tree

RRT* Rapidly-exploring Random Tree Star

RRT*-GBO Rapidly-exploring Random Tree Star with Gradient-Based Optimization

SLERP Spherical Linear Interpolation

SOTA State-of-the-Art

TUM Technische Universität München

List of Algorithms

1.1 `MC_GBO` 32
1.2 `RRT*_GBO` 33
1.3 `connect_to_goal`(x_{last}, x_{goal}) 33
1.4 `neighborhood_query`$(x_{new}, r_{prune}, tree)$ 33
1.5 `rewire_query`$(x_{new}, \boldsymbol{x}_{near})$ 33
1.6 `informed_acceptance_filter` (Optional Variant) 34
1.7 `P_HO` 34
1.8 `sampleAttempt` 34
1.9 `IPSO_solve` 35
1.10 `IPSO_GBO` 35

List of Figures

1.1 Rotation of the body frame $\{\hat{\mathbf{B}}_x, \hat{\mathbf{B}}_y, \hat{\mathbf{B}}_z\}$ by angle α about the z-axis to the inertial frame $\{\hat{\mathbf{I}}_x, \hat{\mathbf{I}}_y, \hat{\mathbf{I}}_z\}$. 4

1.2 Initial frame a, rotated frame b, unit rotation axis $\hat{\mathbf{u}}$, rotation angle θ, and unit quaternion rotation $\mathbf{q}_{ba}$. 7

1.3 Modified Denavit–Hartenberg (MDH) parameters for consecutive link frames. 11

1.4 1-D cubic B-spline trajectory demonstrating the local support property and control vertex influence at an example point $t = 0.55\,\mathrm{s}$. The active knot span $[t_7, t_8] = [0.50, 0.62]\,\mathrm{s}$ dictates that the curve value is determined by control vertices $\mathbf{c}_4$ through $\mathbf{c}_7$, with basis weights $N_{i,3}$ corresponding to active basis functions. Basis weights vary depending on the value of the basis functions for the evaluated point t, respective only to its location within the active knot span. Clamping the knot vector ($\{t_0, \ldots, t_3\}$ and $\{t_{11}, \ldots, t_{14}\}$) anchors the curve endpoints to $\mathbf{c}_0$ and $\mathbf{c}_{10}$, while repeating the initial and final three control vertex values enforces zero derivatives up to second order ($\dot{\mathbf{q}} = \mathbf{0}, \ddot{\mathbf{q}} = \mathbf{0}$) at the boundaries. The grey shaded region indicates local convex hull bounds [5, 10]. 15

1.5 Comparison of uniform and naïve quaternion sampling. The spatial projection, axis–angle distribution, and empirical rotation-angle distribution illustrate the effect of sampling method on the resulting distribution over SO(3). 21

List of Tables

References

[1] Andersson, Joel A. E. et al. "CasADi: A Software Framework for Nonlinear Optimization and Optimal Control." In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36. DOI: 10.1007/s12532-018-0139-4.

[2] Betts, John T. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. 2nd ed. SIAM, 2010.

[3] Boor, Carl de. *A Practical Guide to Splines*. Vol. 27. Applied Mathematical Sciences. New York, NY: Springer-Verlag, 1978.

[4] Craig, John J. *Introduction to Robotics: Mechanics and Control*. 3rd ed. Upper Saddle River, NJ: Pearson/Prentice Hall, 2005, p. 400. ISBN: 0-201-54361-3.

[5] Diebel, James. *Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors*. Tech. rep. Stanford University, Oct. 2006.

[6] Kuffner, James J. "Effective Sampling and Distance Metrics for 3D Rigid Body Path Planning." In: *Proceedings. 2004 IEEE International Conference on Robotics and Automation*. Vol. 4. 2004, pp. 3993–3998. DOI: 10.1109/ROBOT.2004.1308895.

[7] Lampariello, Roberto et al. "Trajectory Planning for Optimal Robot Catching in Real-Time." In: *2011 IEEE International Conference on Robotics and Automation*. 2011, pp. 3719–3726. DOI: 10.1109/ICRA.2011.5980114.

[8] LaValle, Steven M. *Planning Algorithms*. Cambridge, UK: Cambridge University Press, 2006.

[9] Pardis, Shayan, Chignoli, Matthew, and Kim, Sangbae. "Probabilistic Homotopy Optimization for Dynamic Motion Planning." In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. DOI: 10.1109/IROS58592.2024.10802528.

[10] Rao, Anil V. "A Survey of Numerical Methods for Optimal Control." In: *Advances in the Astronautical Sciences* 135.1 (2009), pp. 497–528.

[11] Schuster, Malcolm D. "A Survey of Attitude Representations." In: *The Journal of the Astronautical Sciences* 41.4 (1993), pp. 439–517.

[12] Shoemake, Ken. "Uniform Random Rotations." In: *Graphics Gems III*. Ed. by Kirk, David. Boston, MA: Academic Press, 1992. Chap. III.6, pp. 124–132. ISBN: 0-12-409671-9.

[13] Spiller, Dario, Ansalone, Luigi, and Curti, Fabio. "Particle Swarm Optimization for Time-Optimal Spacecraft Reorientation with Keep-Out Cones." In: *Journal of Guidance, Control, and Dynamics* 39.2 (2016), pp. 312–325. DOI: 10.2514/1.G001228.

[14] Spiller, Dario and Curti, Fabio. “Inverse-Dynamics Particle Swarm Optimization for Real Time Optimal Control: Challenges and Opportunities.” In: *69th International Astronautical Congress (IAC)*. Paper IAC-18,C1,IP,19,x47444. 2018.

[15] Stoneman, Samantha and Lampariello, Roberto. “Embedding Nonlinear Optimization in RRT* for Optimal Kinodynamic Planning.” In: *2014 IEEE 53rd Annual Conference on Decision and Control (CDC)*. 2014, pp. 3737–3744.

[16] Wächter, Andreas and Biegler, Lorenz T. “On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming.” In: *Mathematical Programming* 106.1 (2006), pp. 25–57. DOI: 10.1007/s10107-004-0559-y.